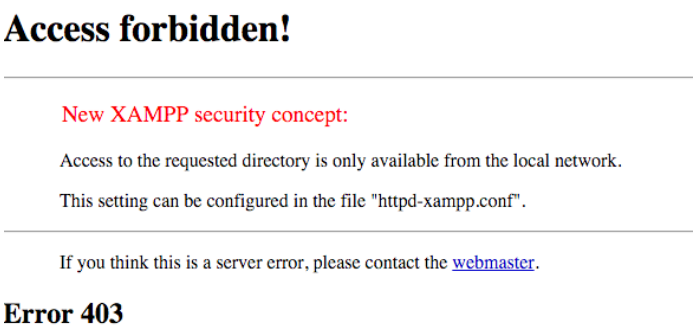
Enable Remote Access to phpMyAdmin

XAMPP includes phpMyAdmin, an open-source, browser-based tool for managing MySQL/MariaDB database servers. By default, for security reasons, phpMyAdmin is configured to only accept connections from the host on which XAMPP is installed. Attempting to access phpMyAdmin from any other host will produce the error below:



To enable remote access to phpMyAdmin from other hosts, follow these steps:

1. Edit the *apache/conf/extra/httpd-xampp.conf* file in your XAMPP installation directory (usually, *C:\xampp*).
2. Within this file, find the block below:

   ```
   <Directory "/xampp/phpMyAdmin">
     AllowOverride AuthConfig
     Require local
     ...
   ```

   Update this block and replace *Require local* with *Require all granted*, so that it looks like this:

   ```
   <Directory "/xampp/phpMyAdmin">
     AllowOverride AuthConfig
     Require all granted
     ...
   ```

3. Save the file and restart the Apache server using the XAMPP control panel.

You should now be able to access phpMyAdmin from other hosts.

Activate and Use Xdebug

Xdebug is a powerful open source debugger and profiler for PHP. It is included with XAMPP and can be used to display stack traces, analyze code coverage and profile your PHP code.

To activate Xdebug, follow these steps:

1. Edit the *php.ini* file in the *php*\ subdirectory of your XAMPP installation directory (usually, *C:\xampp*). Within this file, find the [XDebug] section and within it, add the following configuration directive (or modify the existing one):

```
zend_extension = "C:/xampp/php/ext/php_xdebug.dll"
```

2. Restart the Apache server using the XAMPP control panel.

Xdebug should now be active. To verify this, browse to the URL http://localhost/xampp/phpinfo.php, which displays the output of the *phpinfo()* command. Look through the script and verify that the Xdebug extension is now active.

xdebug

xdebug support	enabled
Version	2.2.3
IDE Key	[redacted]

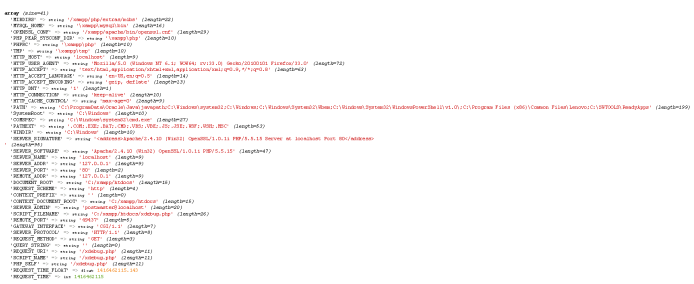
Supported protocols	Revision
DBGp - Common DeBuGger Protocol	$Revision: 1.145 $

Directive	Local Value	Master Value
xdebug.auto_trace	Off	Off
xdebug.cli_color	0	0
xdebug.collect_assignments	Off	Off
xdebug.collect_includes	On	On
xdebug.collect_params	0	0
xdebug.collect_return	Off	Off
xdebug.collect_vars	Off	Off
xdebug.coverage_enable	On	On
xdebug.default_enable	On	On
xdebug.dump.COOKIE	*no value*	*no value*
xdebug.dump.ENV	*no value*	*no value*
xdebug.dump.FILES	*no value*	*no value*
xdebug.dump.GET	*no value*	*no value*

Xdebug overloads the default *var_dump()* function with its own version that includes (among other things) color coding for different PHP types, so you can see it in action immediately by using the *var_dump()* function in a PHP script. For example, create a simple PHP script in the *htdocs*\ subdirectory of your XAMPP installation directory with the following content:

```
<?php
var_dump($_SERVER);
```

When you view your script through a browser, here's an example of what you might see:



One of Xdebug's most powerful features is its ability to profile a PHP script and produce detailed statistics on how long each function call or line of code takes to execute. This can be very useful for performance analysis of complex scripts. To turn on script profiling, follow these steps:

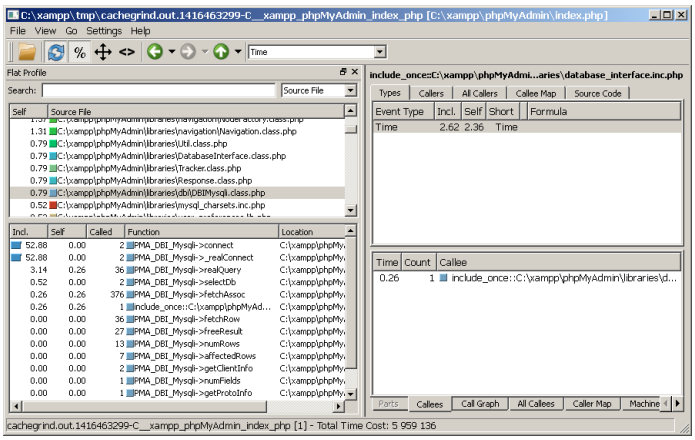
1. Edit the *php.ini* file in the *php*\ subdirectory of your XAMPP installation directory. Within this file, find the [XDebug] section and within it, uncomment and modify the lines below so that they look like this:

```
xdebug.profiler_append = 0
xdebug.profiler_enable = 1
xdebug.profiler_enable_trigger = 0
xdebug.profiler_output_dir = "C:/xampp/tmp"
xdebug.profiler_output_name = "cachegrind.out.%t-%s"
```

2. Restart the Apache server using the XAMPP control panel.

At this point, Xdebug profiling is active. Every PHP script that you run will be profiled and the results will be placed in the *C:\xampp\tmp* as a so-called cachegrind file. You can view this cachegrind file with a tool like WinCacheGrind, which you must download and install separately.

To illustrate how this works, consider the screenshot below, which shows the profiled output of a script using WinCacheGrind. As the screenshot illustrates, it's easy to see the entire life cycle of a PHP script, including the call sequence and the amount of time taken by each function, and thereby find targets for further optimization.

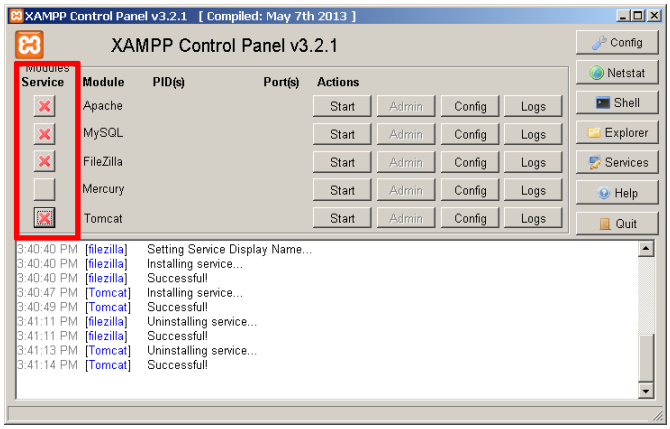


TIP To find out more about Xdebug's powerful features, read the Xdebug documentation.

Auto-Start XAMPP

By default, you have to start XAMPP or its components manually every time you reboot your system. However, you can also configure XAMPP components to start automatically on system boot. Follow the steps below.

1. Launch the XAMPP control panel.
2. Stop all the running XAMPP components by clicking the "Stop" button next to each component.
3. Click the "Service" button next to each component to install it as a service. Click "Yes" when prompted to confirm. This will install the corresponding service and set it to start automatically on boot.



4. Reboot your system and the selected components should start automatically.

To revert the changes, simply uncheck the selected services in the XAMPP control panel.

Backup and Restore MySQL/MariaDB Databases

As you use XAMPP, you might find that you need to backup or restore a MySQL or MariaDB database. There are two easy ways to do this with XAMPP: using the browser-based phpMyAdmin tool or using MySQL/MariaDB's command-line tools. This guide describes both options.

This guide assumes that you already have a PHP application that uses a MySQL/MariaDB database deployed on XAMPP. The example application used in this guide is WordPress, although the steps outlined below will work for any application.

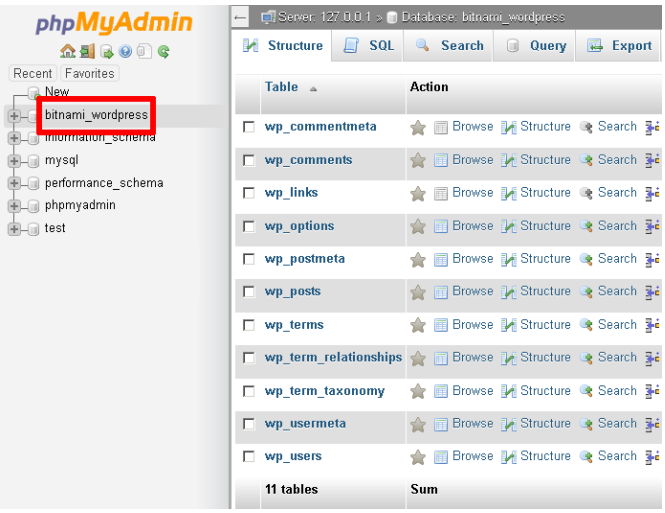
TIP Remember that you can install WordPress quickly using the Bitnami WordPress module for XAMPP.

Using phpMyAdmin

XAMPP includes phpMyAdmin, an open-source, browser-based tool for managing MySQL/MariaDB database servers. To backup a MySQL/MariaDB database using phpMyAdmin, follow these steps:

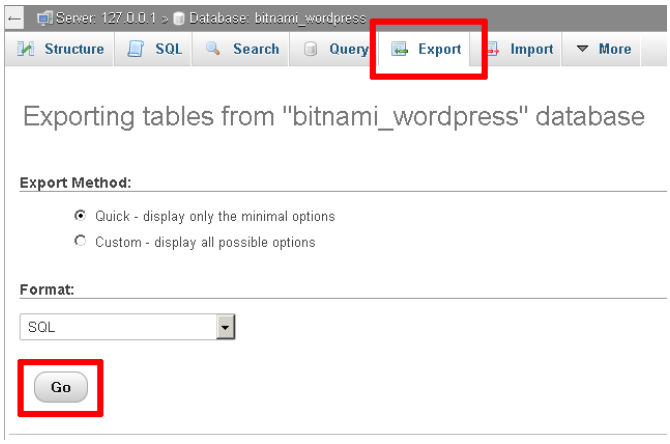
1. Browse to http://localhost/phpMyAdmin or http://127.0.0.1/phpMyAdmin. If required, log in using your database access credentials. On a fresh XAMPP installation without any changes, you can log in as *root* with a blank password.

2. Select the database to be backed up from the list in the left navigation pane. In this case, we're backing up the WordPress database, which is named *bitnami_wordpress*.

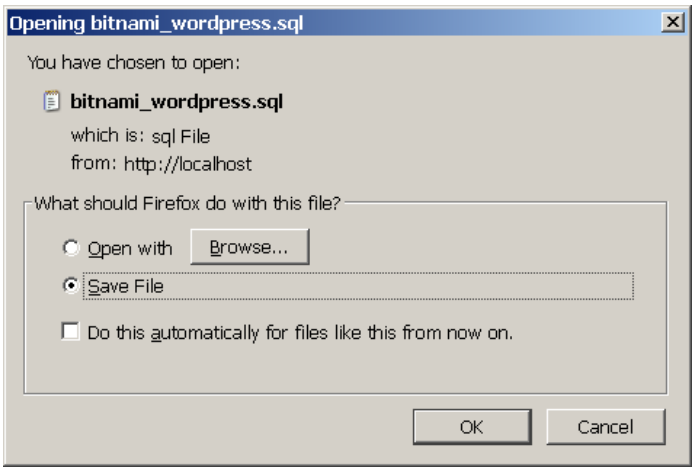


3. Select the "Export" command in the top navigation bar.

4. On the resulting page, select "Quick" as the export method and "SQL" as the output format. Click "Go" to proceed.

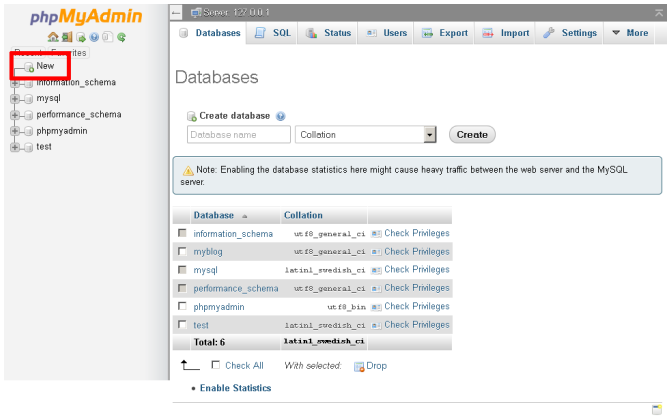


phpMyAdmin will export the database to a text file as a series of SQL statements. Once done, the browser will prompt you to download it to the desktop. This text file is your backup, so store it safely!



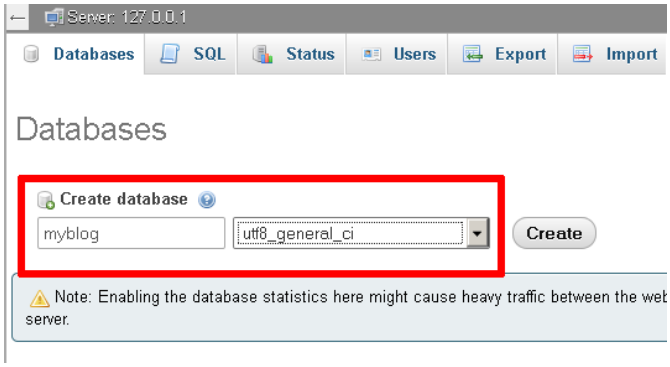
At a later point, you may wish to restore the database. To restore the data to a fresh MySQL/MariaDB database using phpMyAdmin, follow these steps:

1. Browse to http://localhost/phpMyAdmin or http://127.0.0.1/phpMyAdmin. If required, log in using your database access credentials.
2. Select the "New" command in the left navigation pane.

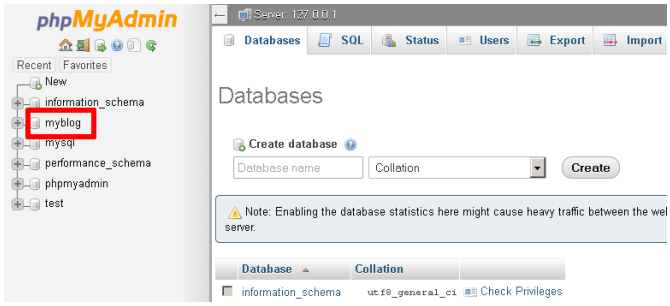


3. On the resulting page, enter a name for the new database (in this case, *myblog*). Select the collation

"utf8_general_ci". Click "Create" to create the database.

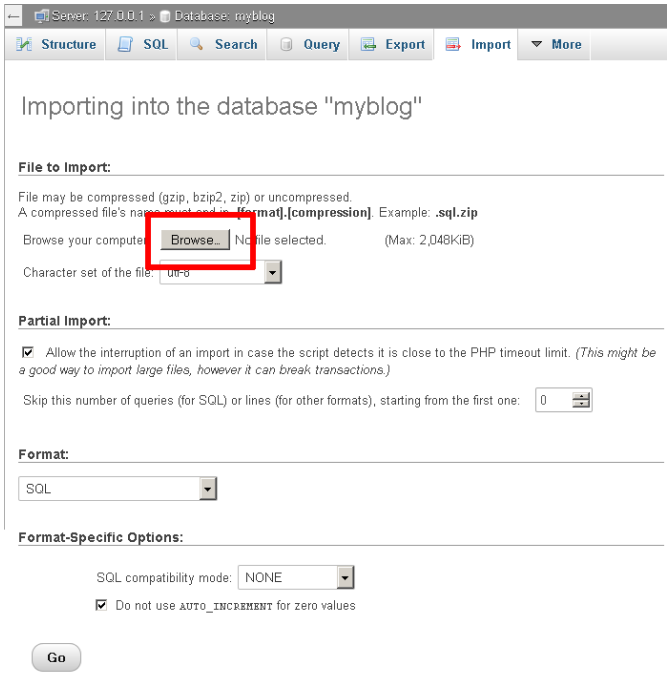


Once the database has been created, it will appear in the left navigation pane.

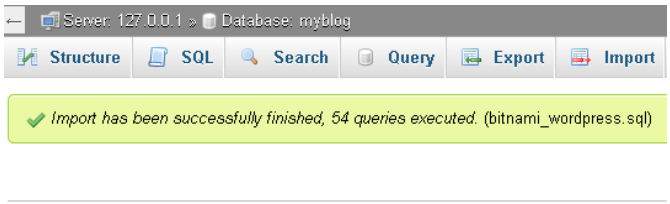


4. Select the new database from the left navigation pane. In the resulting page, select the "Import" command in the top navigation bar.

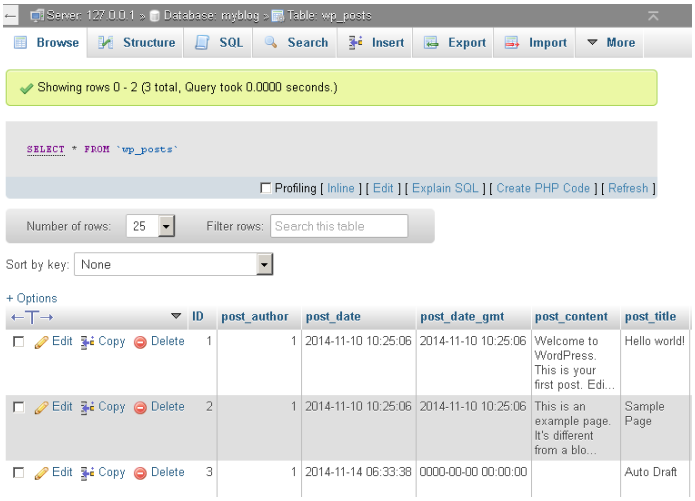
5. Click the "Browse…_" button and select the backup file created earlier. Click "Go" to proceed.



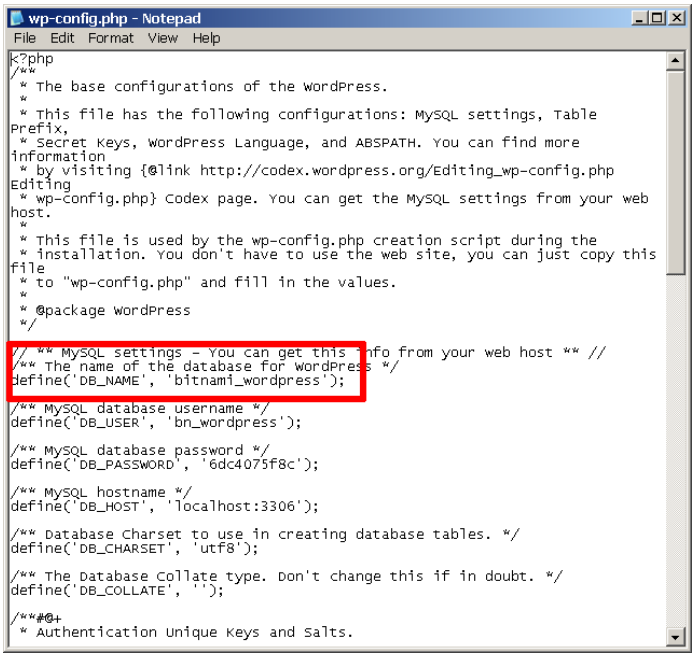
phpMyAdmin will import the data from the backup file. Once complete, you will see a screen indicating the status of the import.



If you browse the contents of the database, you should now see your original data.



You should now update your application's configuration and point it to your newly-created database. For example, with the Bitnami WordPress module, you will need to edit the *wp-config.php* file in the *apps\wordpress\htdocs* subdirectory of your XAMPP installation directory. Within this file, you will need to update the *DB_NAME* constant to use the new database name.



NOTE

By default, PHP (and therefore phpMyAdmin) is configured to only accept uploads up to 2 MB in size. If your database is larger than this, you need to modify the PHP configuration file to accept a larger upload size. To do this, edit the *php.ini* file in the *php*\ subdirectory of your XAMPP installation directory and increase the values of the *post_max_size* and *upload_max_filesize* variables. Save your changes and restart Apache for the changes to take effect.

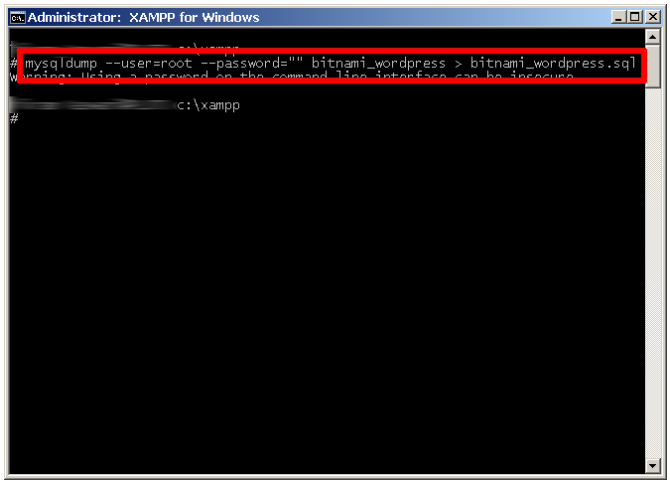
Using Command-Line Tools

MySQL and MariaDB include two command-line tools that you can use to quickly backup and restore databases. The *mysqldump* tool is used to export the contents of a database to a text file, while the *mysql* client can be used to import data from a text file into a MySQL/MariaDB database.

To backup a MySQL/MariaDB database from the command line, follow these steps:

1. Launch a new Windows command shell using the "Shell" button in the XAMPP control panel.
2. Use the command below to export the contents of the selected database. In this example, we're backing up the WordPress database, which is named *bitnami_wordpress*, to a file named *bitnami_wordpress.sql*. This text file is your backup, so store it safely!

   ```
   mysqldump --user=root --password="" bitnami_wordpress > bitnami_wordpress.sql
   ```

At a later point, you may wish to restore the database. To restore the data to a fresh MySQL/MariaDB database from the command line, follow these steps:

1. Launch a new Windows command shell using the "Shell" button in the XAMPP control panel.
2. Use the *mysql* client to create a new, empty database to hold your data. In this example, the new database is named *myblog*.

   ```
   mysql --user=root --password="" -e "CREATE DATABASE myblog"
   ```

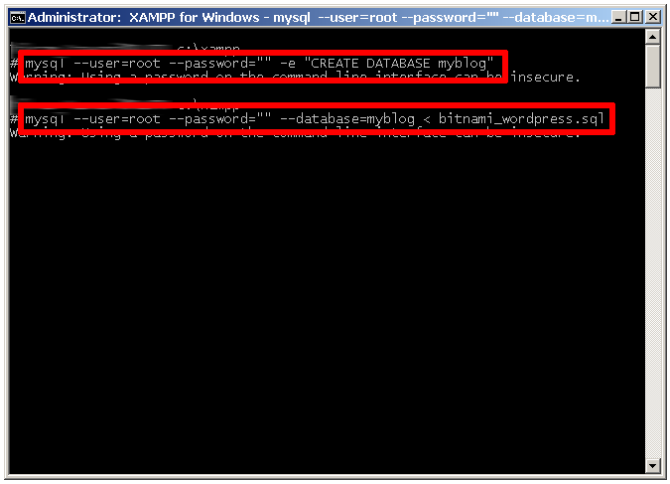
   Remember to use the correct database access credentials in the command. On a fresh XAMPP installation

without any changes, you can usually log in as root with a blank password.

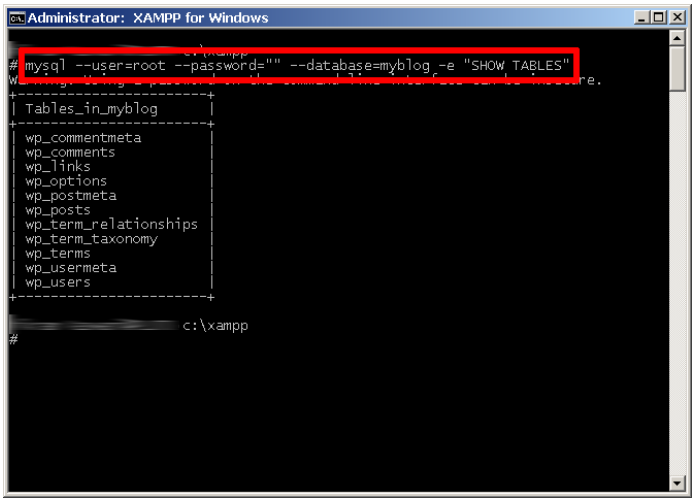
3. Use the *mysql* client to import the contents of the backup file into the new database.

```
mysql --user=root --password="" --database=myblog < bitnami_wordpress.sql
```

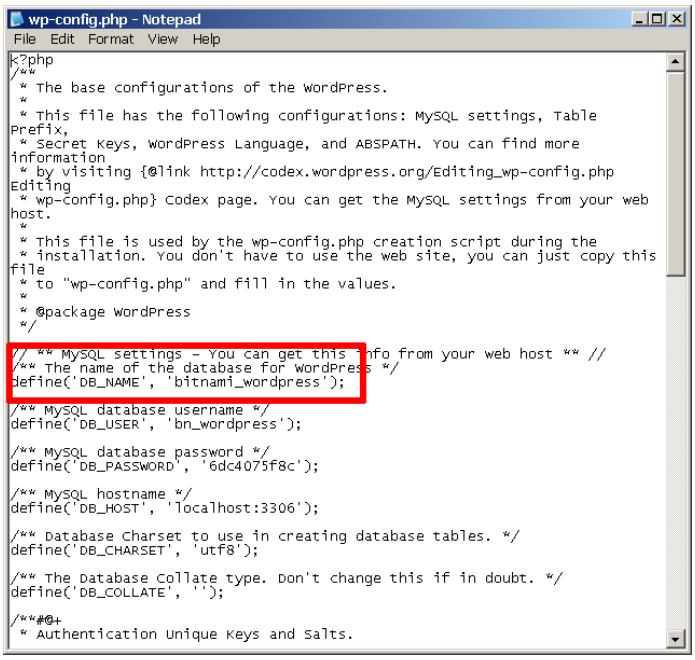
The command-line client will now import the data from the backup file.



If you browse the contents of the database, you should now see your original data.



You should now update your application's configuration and point it to your newly-created database. For example, with the Bitnami WordPress module, you will need to edit the *wp-config.php* file in the *apps\wordpress\htdocs* subdirectory of your XAMPP installation directory. Within this file, you will need to update the *DB_NAME* constant to use the new database name.


wp-config.php - Notepad
File Edit Format View Help
k?php
/**
* The base configurations of the WordPress.
*
* This file has the following configurations: MySQL settings, Table
Prefix,
* Secret Keys, WordPress Language, and ABSPATH. You can find more
information
* by visiting {@link http://codex.wordpress.org/Editing_wp-config.php
Editing
* wp-config.php} Codex page. You can get the MySQL settings from your web
host.
*
* This file is used by the wp-config.php creation script during the
* installation. You don't have to use the web site, you can just copy this
file
* to "wp-config.php" and fill in the values.
*
* @package WordPress
*/
// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'bitnami_wordpress');
/** MySQL database username */
define('DB_USER', 'bn_wordpress');
/** MySQL database password */
define('DB_PASSWORD', '6dc4075f8c');
/** MySQL hostname */
define('DB_HOST', 'localhost:3306');
/** Database Charset to use in creating database tables. */
define('DB_CHARSET', 'utf8');
/** The Database Collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');
/**#@+
* Authentication Unique Keys and Salts.

Change the MySQL Temporary Files Directory

By default, MySQL's temporary files are stored in the *tmp*\ subdirectory of your XAMPP installation directory (usually, *C:\xampp*). In case you wish to change the location of this directory, follow these steps:

1. Edit the *my.ini* file in the *mysql**bin*\ subdirectory of your XAMPP installation directory. Within this file, find the *tmpdir* variable in the [mysqld] section and change it to reflect your new desired location. For example:

   ```
   tmpdir = "D:/temp"
   ```

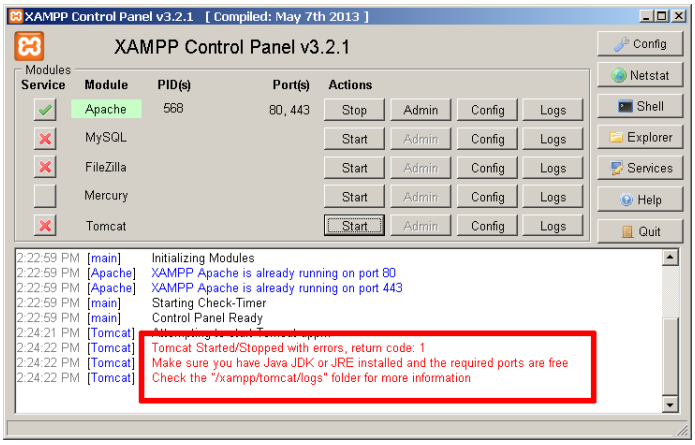
2. Save your changes to the file.
3. Restart the MySQL server using the XAMPP control panel.

MySQL's temporary files should now get created in the new location.

Configure and Use Apache Tomcat

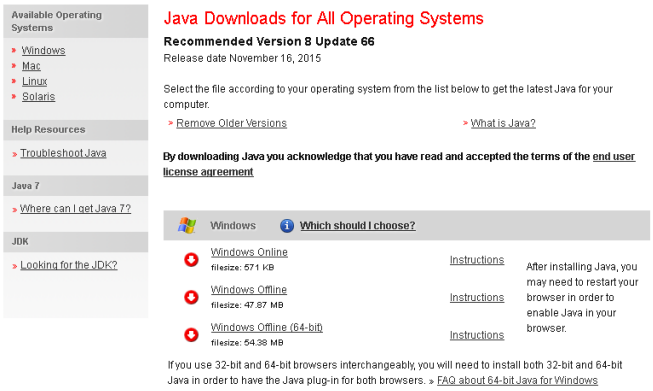
XAMPP for Windows comes with Apache Tomcat built in, making it easy to get started with Java-based Web applications. This guide will walk you through the process of using Apache Tomcat with XAMPP.

XAMPP for Windows does not include Java, which is a pre-requisite for using Apache Tomcat. If you do not already have Java installed, you will see an error similar to the below when you attempt to start Apache Tomcat through the XAMPP control panel.



To correct this, install Java and then attempt to use Apache Tomcat, as below:

1. Browse to https://www.java.com/en/download/ and follow the instructions to download the latest version of the Java Runtime Environment.



2. Follow the on-screen instructions to install Java.



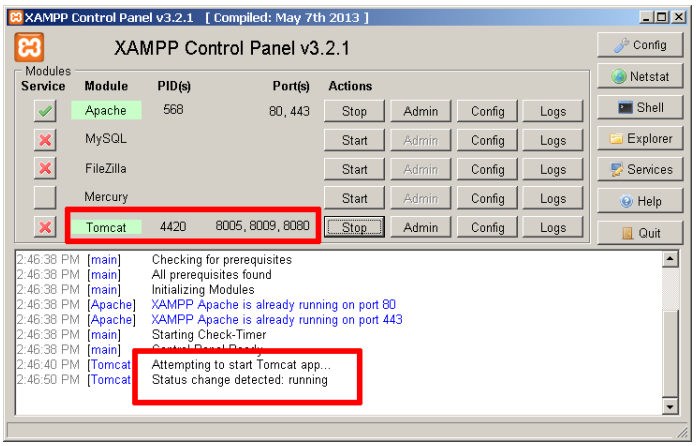
Once installed, you will see a message like the one below:



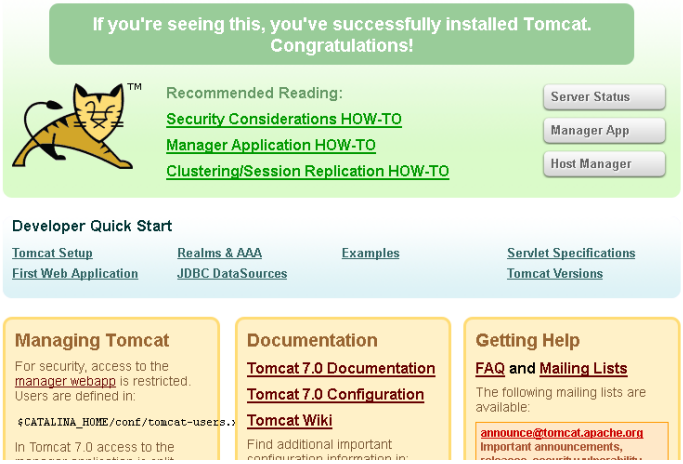
NOTE

On some versions of Windows, you will not be able to install Java without administrator privileges. You may either need to log in as a user with administrative rights or, if you are using the offline Java installer, you may need to right-click the installer file and select the "Run as administrator" menu option, then enter administrator credentials (if required) to proceed.

3. Start the Apache Tomcat server using the XAMPP control panel.



You should now be able to access Apache Tomcat by browsing to http://localhost:8080/ in your browser's address bar. Here's an example of what you should see:



Apache Tomcat includes two applications, the "manager" application and the "host-manager" application, that simplify management and deployment of Web applications and provide detailed information on server status. These applications can be accessed from the Apache Tomcat welcome page, as highlighted in the image above.

Access to the above applications is blocked by default. To access them, you must configure one or more sets of administrator accounts and then assign the roles "manager-gui" and/or "admin-gui" to these accounts. Accounts with the "manager-gui" role would have access to the "manager" application, and those with the "admin-gui" role would have access to the "host-manager" application.

To configure these accounts, follow the steps below:

1. Edit the *tomcat-users.xml* file in the *tomcat/conf* subdirectory of your XAMPP installation directory (typically *C:\xampp*) and add the line below, before the closing *</tomcat-users>* element:

   ```
   <user username="admin" password="admin" roles="manager-gui,admin-gui" />
   ```

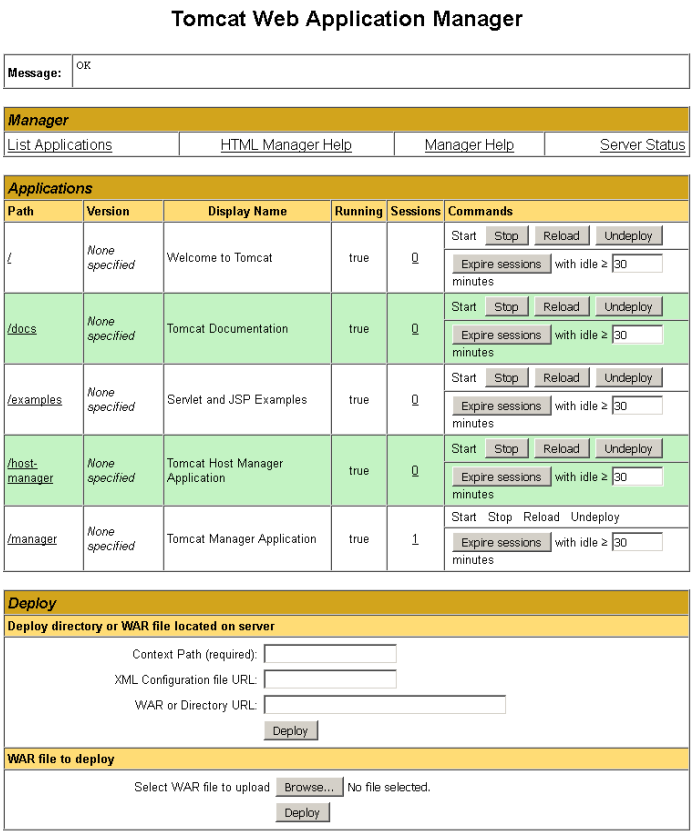
   This configures an account with username "admin" and password "admin", with access to both the Apache Tomcat Web applications. Remember to replace the username and password with values specific to your installation.

If you prefer to have a separate account for each application, replace the previous configuration with this example:

```
<user username="manager" password="manager" roles="manager-gui" />
<user username="admin" password="admin" roles="admin-gui" />
```

2. Save the changes.
3. Restart the Apache Tomcat server using the XAMPP control panel.

You should now be able to access the Apache Tomcat Web applications using the configured credentials. Here's an example of the management Web application, which includes controls to deploy new WAR applications.



Configure Virtual Hosts

Apache makes it easy to have multiple websites or web applications running on the same physical server, and access each using a custom domain name. This is known as *virtual hosting*, and each virtual host can be mapped to a different subdirectory of the server.

This feature is especially useful in development scenarios, as it allows developers to isolate and access different applications using unique domain names. For example, the domains *http://app01.localhost* and *http://app02.localhost* could point to two separate applications on the same server. Or, you could create separate domains, such as *http://client1/*, *http://client2/* and so on, for multiple client websites hosted on the same server.

NOTE

Virtual hosts created in this manner will not be accessible from other systems, unless those systems are separately configured to associate the custom domain used by the virtual host with the IP address of the XAMPP server. This guide walks you through the process of setting up a new virtual host with XAMPP, by creating and configuring an example host named *wordpress.localhost* for a WordPress installation. It assumes that you have an existing WordPress installation at *C:\xampp\apps\wordpress\htdocs* and accessible via the URL http://localhost/wordpress.

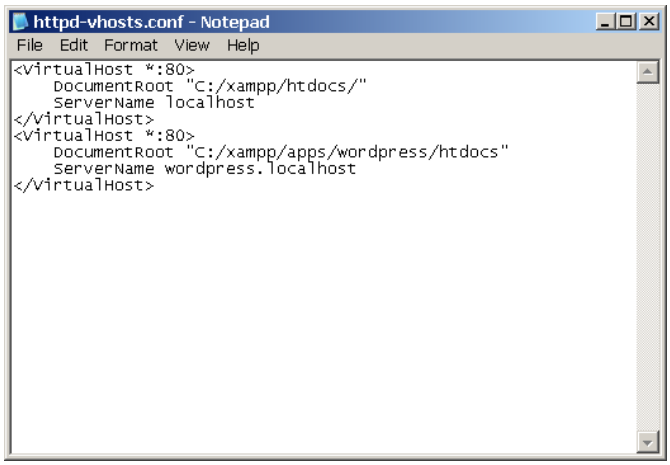
TIP

You can install WordPress with just a few clicks using the Bitnami WordPress Module for XAMPP.

Follow the steps below to create a virtual host:

1. Change to your XAMPP installation directory (typically, *C:\xampp*) and open the *httpd-vhosts.conf* file in the *apache\conf\extra* subdirectory using your favourite text editor.
2. Replace the contents of this file with the following directives:

```
<VirtualHost *:80>
    DocumentRoot "C:/xampp/htdocs/"
    ServerName localhost
</VirtualHost>
<VirtualHost *:80>
    DocumentRoot "C:/xampp/apps/wordpress/htdocs"
    ServerName wordpress.localhost
</VirtualHost>
```



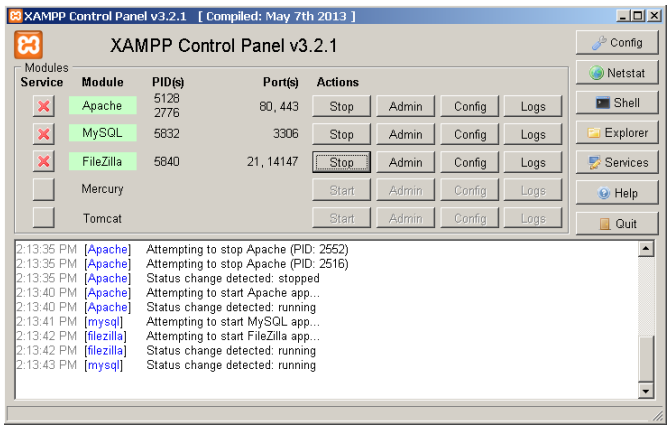
This contains two virtual host configuration blocks:

- The first block is the default or fallback virtual host, which is used for all requests that are not matched by subsequent blocks.
- The second block sets up a virtual host named *wordpress.localhost*. The DocumentRoot directive specifies the directory to be used when serving requests for this virtual host (in this case, the WordPress installation directory), while the ServerName directive specifies the custom domain name for the virtual host.

To add more virtual hosts, simply duplicate the second virtual host block and modify the port number, DocumentRoot and ServerName directives as per your requirements. For example, if you want to use SSL with your custom domain name, you can add a new virtual host block for port 443.

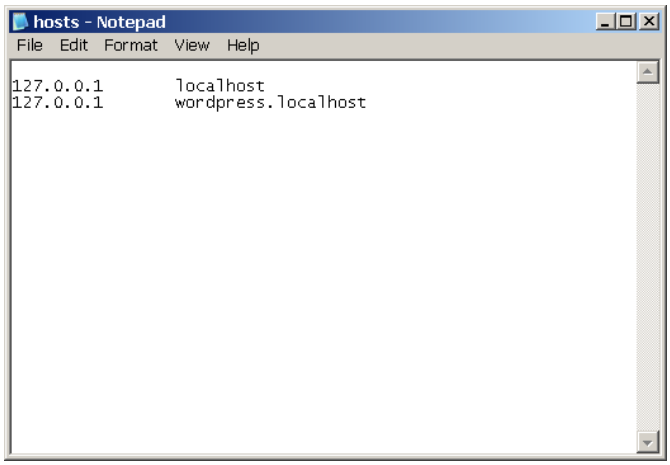
TIP	If you plan to have a large number of virtual hosts with very similar configuration, consider using wildcard-based subdomains with your virtual hosts.

3. Restart Apache using the XAMPP control panel for your changes to take effect.



4. At this point, your virtual host is configured. However, if you try browsing to the *wordpress.localhost* domain, your browser will show a failure notice, since this domain does not exist in reality. To resolve this, it is necessary to map the custom domain to the local IP address. To do this, open the file *C:\windows\system32\drivers\etc\hosts* and add the following line to it:

```
127.0.0.1       wordpress.localhost
```

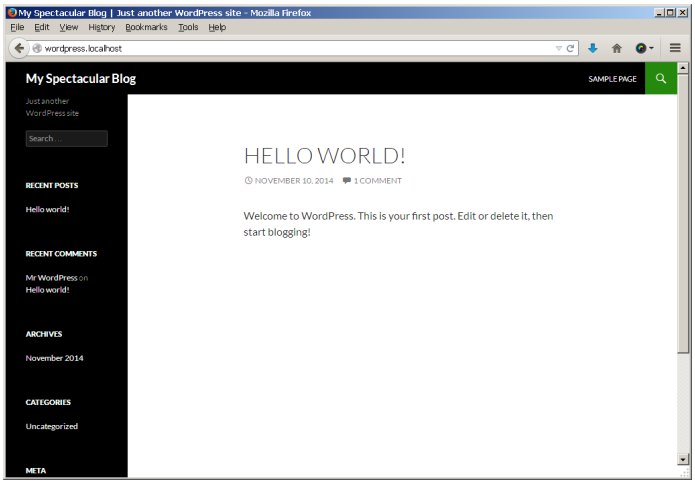


This takes care of mapping the *wordpress.localhost* domain name to the local machine, which always has the IP address 127.0.0.1.

NOTE

On some versions of Windows, you will not be able to edit the *C:\windows\system32\drivers\etc\hosts* file without administrator privileges. You can edit the file by right-clicking the Notepad icon and selected the "Run as administrator" menu option, then entering administrator credentials (if required) and clicking "OK" or "Yes" to proceed.

At this point, you should be able to enter the URL http://wordpress.localhost in your browser's address bar and see your WordPress installation. Here's an example of what you should see:



Configure Wildcard-Based Subdomains

Apache's virtual hosting feature makes it easy to host multiple websites or web applications on the same server, each accessible with a different domain name. However, when you have a large number of virtual hosts sharing almost-identical configuration, wildcard-based subdomains simplify maintenance and reduce the effort involved in adding a new virtual host.

With wildcard subdomains, it's no longer necessary to edit the Apache configuration file or restart the server to initialize a new virtual host. Instead, you simply need to create a subdirectory matching the subdomain name on the server with your content, and Apache will automatically use that directory to serve requests for the corresponding subdomain.

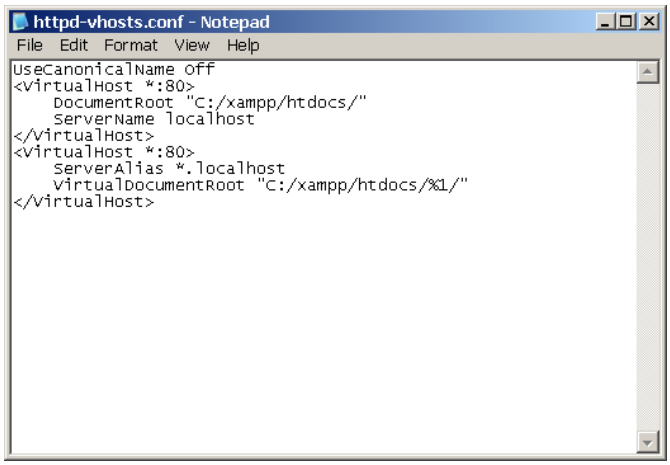
NOTE	Virtual hosts created in this manner will not be accessible from other systems, unless those systems are separately configured to associate the custom domains used by virtual hosts with the IP address of the XAMPP server.

This guide walks you through the process of setting up wildcard virtual hosts with XAMPP, such that requests for *subdomain.localhost* are automatically served by the *subdomain/* directory of the main server document root. Follow the steps below: . Change to your XAMPP installation directory (typically, *C:\xampp*) and open the *httpd.conf* file in the *apache\conf* subdirectory using your favourite text editor. Within the file, find the following line and uncomment it by removing the hash symbol (#) at the beginning of the line.

```
LoadModule vhost_alias_module modules/mod_vhost_alias.so
```

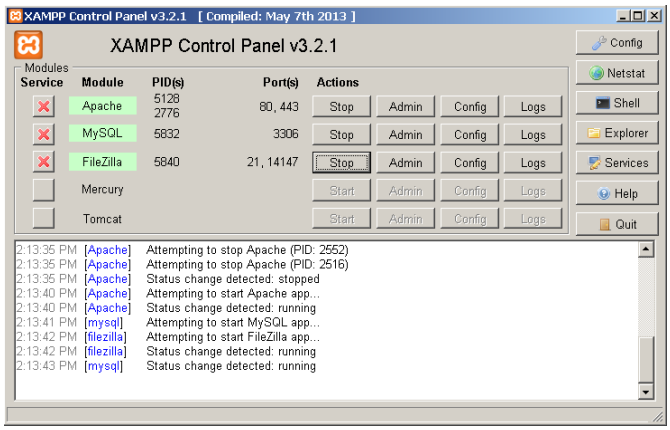
1. Next, edit the *httpd-vhosts.conf* file in the *apache\conf\extra* subdirectory of your XAMPP installation directory.
2. Replace the contents of this file with the following directives:

```
UseCanonicalName Off
<VirtualHost *:80>
    DocumentRoot "C:/xampp/htdocs/"
    ServerName localhost
</VirtualHost>
<VirtualHost *:80>
    ServerAlias *.localhost
    VirtualDocumentRoot "C:/xampp/htdocs/%1/"
</VirtualHost>
```



In this configuration, the first virtual host block configures how requests are handled by default. The second block configures wildcard virtual hosting for subdomains, such that requests for *subdomain.localhost* are automatically served by the *subdomain*\ directory of the *C:\xampp\htdocs*\ directory. In particular, notice the %1 placeholder, which matches the subdomain name from the request URL.

3. Restart Apache using the XAMPP control panel for your changes to take effect.



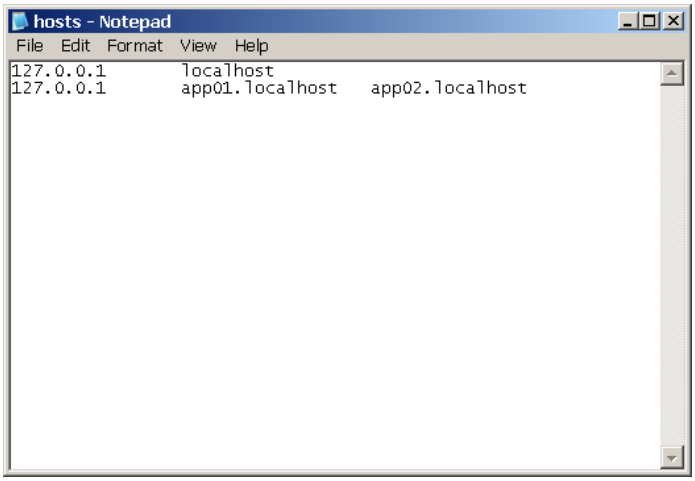
At this point, your wildcard subdomains are configured. You can easily test this by using the Windows Explorer to create two new subdirectories at *C:\xampp\htdocs\app01*\ and *C:\xampp\htdocs\app02*\. Within each subdirectory, create a file named *index.html* and fill it with some sample HTML content. Use different content for each file, so that you can easily distinguish that they're being served from different directories - for example:

```
<!-- index.html in app01 directory -->
<html>
 <head></head>
 <body>
  <h1>Welcome to app01.localhost!</h1>
 </body>
</html>
```

```
<!-- index.html in app02 directory -->
<html>
 <head></head>
 <body>
  <h1>Hello from app02.localhost!</h1>
 </body>
</html>
```

Since these domains do not actually exist in reality, you also need to map them to the local IP address. Open the file *C:\windows\system32\drivers\etc\hosts* in a text editor and add the following line to it:

```
127.0.0.1       app01.localhost   app02.localhost
```



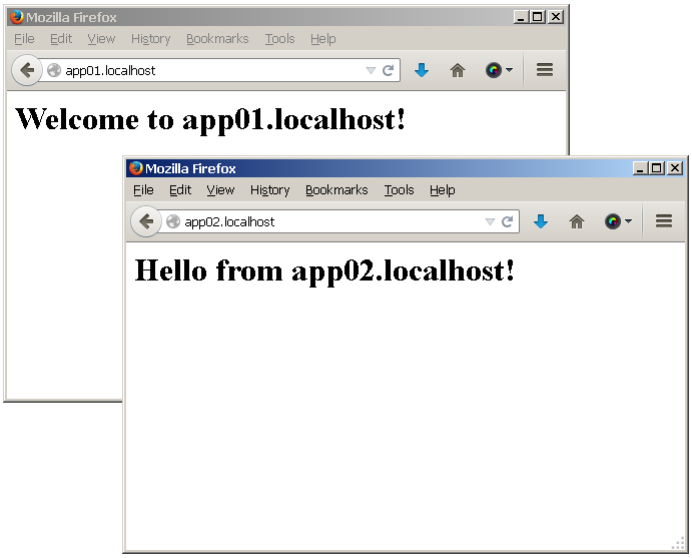
TIP

You will need to do this every time you want to configure a new wildcard virtual host, because the Windows *hosts* file does not support wildcard entries. To avoid this step, you can use a DNS server or local DNS proxy that supports wildcards and therefore takes care of automatically resolving requests for **.localhost* to the local IP address.

NOTE

On some versions of Windows, you will not be able to edit the *C:\windows\system32\drivers\etc\hosts* file without administrator privileges. You can edit the file by right-clicking the Notepad icon and selected the "Run as administrator" menu option, then entering administrator credentials (if required) and clicking "OK" or "Yes" to proceed.

At this point, you should be able to enter the URLs http://app01.localhost or http://app02.localhost in your browser's address bar, and you should then see the corresponding HTML page.


Mozilla Firefox
File Edit View History Bookmarks Tools Help
app01.localhost
Welcome to app01.localhost!
Mozilla Firefox
File Edit View History Bookmarks Tools Help
app02.localhost
Hello from app02.localhost!

Start a New Zend Framework 1 Project

XAMPP makes it easy to start developing with PHP, and Zend Framework is one of the most popular PHP development frameworks. This guide walks you through the process of initializing a new Zend Framework 1 project with XAMPP.

> **NOTE** This guide assumes that the new Zend Framework application will be accessible at the URL http://localhost/myapp/.

1. Download the latest version of the Zend Framework 1 minimal package as a ZIP archive.
2. Within your XAMPP installation directory (usually *C:\xampp*), create a new directory named *apps*\ (if it doesn't already exist). Then, within this new *apps*\ directory, create a directory to hold your Zend Framework application and its related XAMPP configuration files. In this example, call the directory *myapp*\.
3. Extract the contents of the ZIP archive to the *myapp*\ *directory. This will produce a _ZendFramework-[x.y]-minimal.zip*\ subdirectory in the *myapp*\ directory. Rename this newly-created subdirectory to *htdocs*.

   > **NOTE** This new *htdocs* directory will be the main working directory for your Zend Framework project.

4. Within the *myapp*\ directory, create a new *conf*\ subdirectory.
   a. Within the new *conf*\ subdirectory, use your text editor to create and populate a file named *ttpd-prefix.conf* with the following content:

   ```
   Alias /myapp/ "C:/xampp/apps/myapp/htdocs/public/"
   Alias /myapp "C:/xampp/apps/myapp/htdocs/public"
   Include "C:/xampp/apps/myapp/conf/httpd-app.conf"
   ```

   b. Within the *conf*\ subdirectory, also create and populate a file named *httpd-app.conf* with the following content:

   ```
   <Directory C:/xampp/apps/myapp/htdocs/public>
     Options +FollowSymLinks
     AllowOverride All
     Require all granted
   </Directory>
   ```

5. Edit the *httpd-xampp.conf* file in the *apache**conf**extra*\ subdirectory of your XAMPP installation directory and add the following line at the end to include the *httpd-prefix.conf* file created earlier.

   ```
   Include "C:/xampp/apps/myapp/conf/httpd-prefix.conf"
   ```

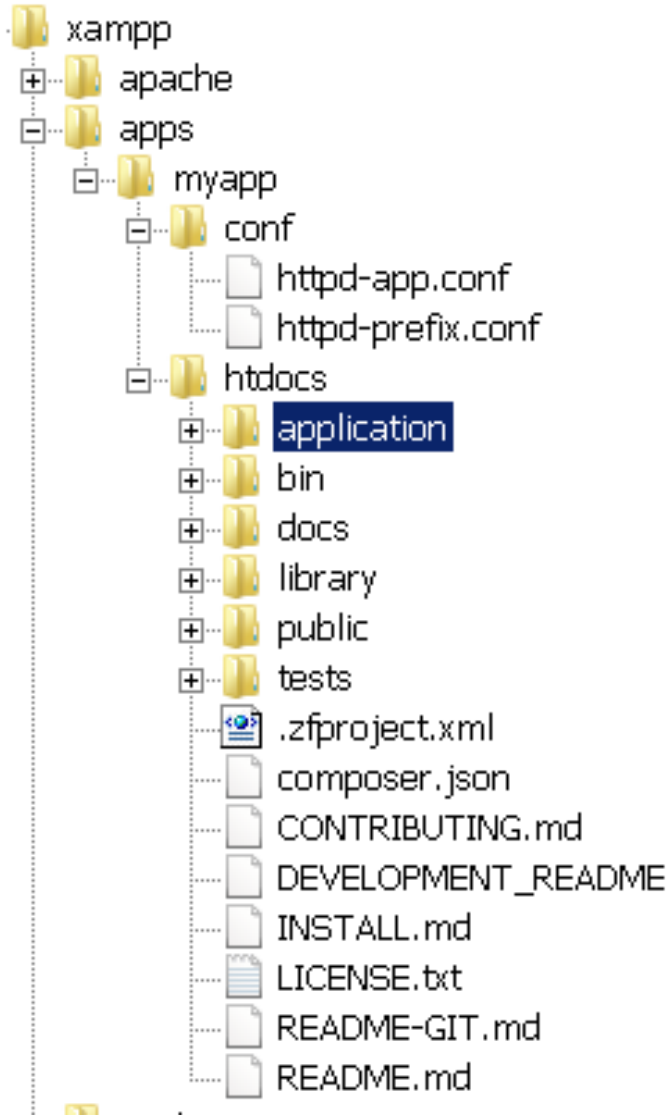
   > **NOTE** Remember to update the above file and directory paths so that they're valid for your system.

6. Launch a new Windows command shell using the "Shell" button in the XAMPP control panel.

7. Within the command shell, change to the *myapp\htdocs* directory and run the following commands to create a new stub project.

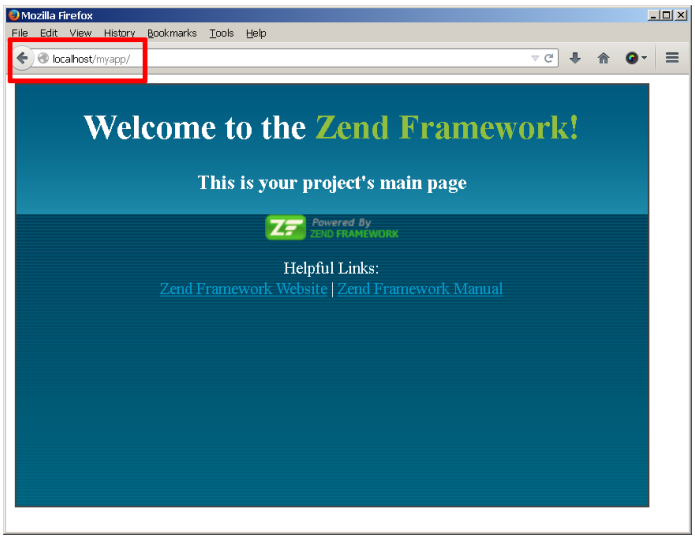
```
bin\zf.bat create project .
```

8. Check that you have a directory structure like this:



9. Restart the Apache server using the XAMPP control panel.

You should be able to access the Zend Framework skeleton application by browsing to http://localhost/myapp. Here's what the default welcome page looks like:



You can now begin developing your Zend Framework application by modifying the skeleton application code. For more information, refer to the Zend Framework documentation.

Start a New Zend Framework 2 Project

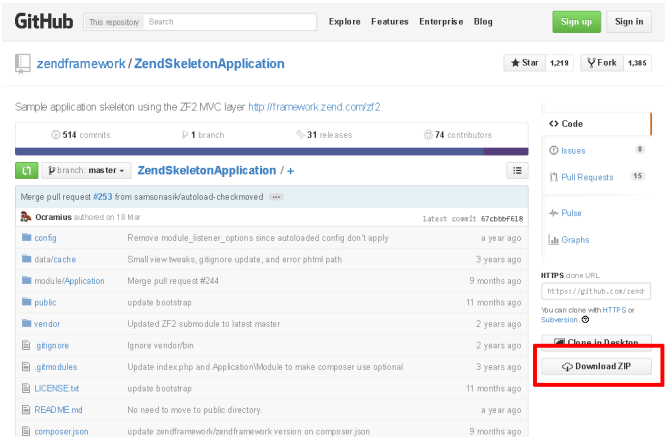
XAMPP makes it easy to start developing with PHP, and Zend Framework 2 is one of the most popular PHP development frameworks. This guide walks you through the process of initializing a new Zend Framework 2 project with XAMPP.

NOTE

This guide assumes that you have already downloaded and installed Git for Windows, and that the git binary is in your Windows PATH. If you don't have Git for Windows, you can download and install it from here. It also assumes that the new Zend Framework 2 application will be accessible at the URL http://localhost/myapp/.

Follow these steps:

1. Download the Zend Framework 2 skeleton application as a ZIP archive from Github.



2. Within your XAMPP installation directory (usually *C:\xampp*), create a new directory named *apps*\ (if it doesn't already exist). Then, within this new *apps*\ directory, create a directory to hold your Zend Framework 2 application and its related XAMPP configuration files. In this example, call the directory *myapp*\.

3. Extract the contents of the ZIP archive to the *myapp*\ directory. This will produce a *zendframework-ZendSkeletonApplication-[xx]*\ subdirectory in the *myapp*\ directory. Rename this newly-created subdirectory to *htdocs*.

NOTE

This new *htdocs* directory will be the main working directory for your Zend Framework 2 project.

4. Within the *myapp*\ directory, create a new *conf*\ subdirectory.

   a. Within the new *conf*\ subdirectory, use your text editor to create and populate a file named *httpd-prefix.conf* with the following content:

```
Alias /myapp/ "C:/xampp/apps/myapp/htdocs/public/"
Alias /myapp "C:/xampp/apps/myapp/htdocs/public"
Include "C:/xampp/apps/myapp/conf/httpd-app.conf"
```

   b. Within the *conf*\ subdirectory, also create and populate a file named *httpd-app.conf* with the following

content:

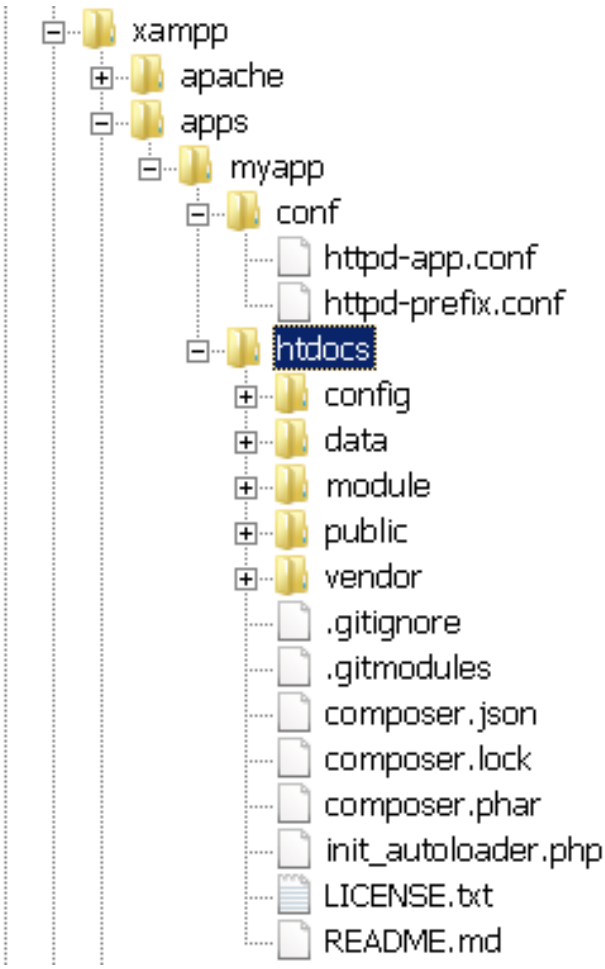
```
<Directory C:/xampp/apps/myapp/htdocs/public>
  Options +FollowSymLinks
  AllowOverride All
  Require all granted
</Directory>
```

5. Edit the *httpd-xampp.conf* file in the *apache\conf\extra* subdirectory of your XAMPP installation directory and add the following line at the end to include the *httpd-prefix.conf* file created earlier.

```
Include "C:/xampp/apps/myapp/conf/httpd-prefix.conf"
```

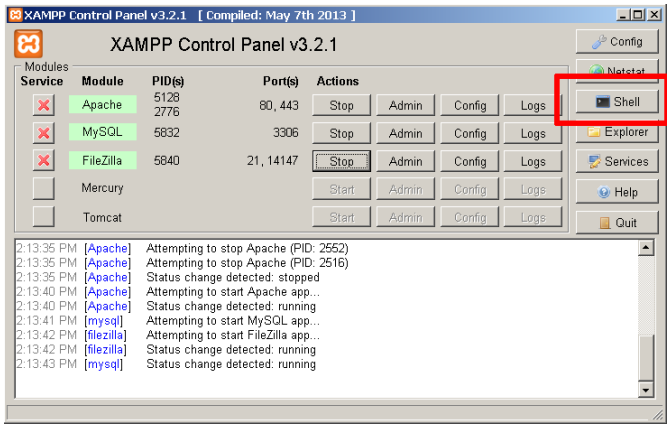
> **NOTE** Remember to update the above file and directory paths so that they're valid for your system.

6. Check that you have a directory structure like this:



7. Restart the Apache server using the XAMPP control panel.

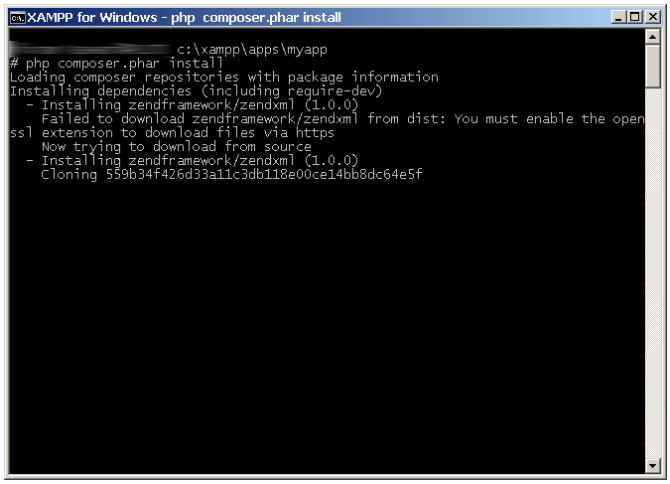
8. Launch a new Windows command shell using the "Shell" button in the XAMPP control panel.



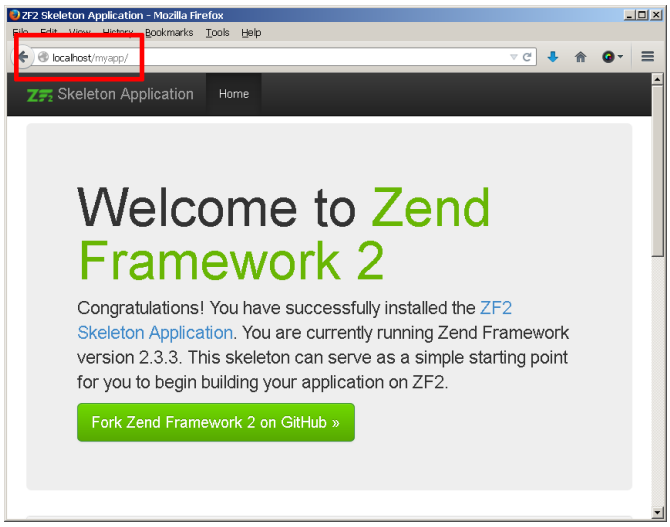
9. Within the command shell, change to the *myapp\htdocs* directory and run the following commands to update Composer (the PHP dependency manager) and install the Zend Framework 2 components.

```
php composer.phar self-update
php composer.phar install
```

Here's an example of what you might see as Composer downloads and installs dependencies.



10. Once Composer is done, you should be able to access the Zend Framework 2 skeleton application by browsing to http://localhost/myapp. Here is an example of the default welcome page you should see:



You can now begin developing your Zend Framework 2 application by modifying the skeleton application code. For more information, refer to the Zend Framework 2 User Guide.

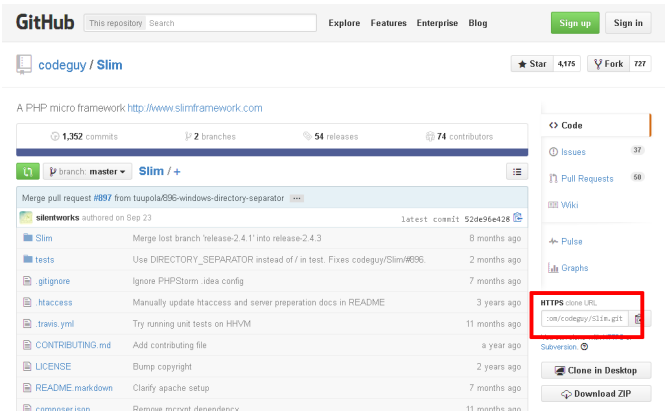
Deploy an Application Using Git

Git is a popular distributed version control system. As you work with XAMPP, you might often find yourself needing to download and install applications or code libraries from a developer's public git repository.

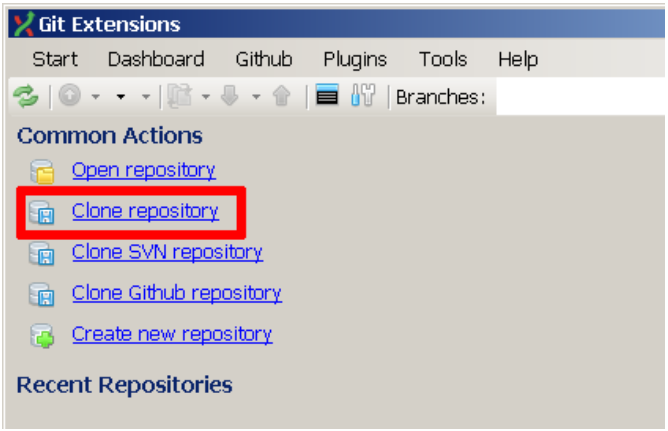
This guide walks you through the process, showing you how to use a git client to transfer code from a public online git repository to your XAMPP server. As an example, this guide assumes that you wish to download the latest version of Slim, a popular PHP micro-framework for Web application development that also has its own Github repository. NOTE: This guide uses Git Extensions, an open source git client that's available for Windows, Linux and Mac OS, and assumes that you have already downloaded and installed this software. If you don't already have this, you can download it from the official Git Extensions website, and install it using these instructions.

To download Slim (or any other application) from its git repository, follow these steps:

1. Ensure that your Apache server is running.
2. Obtain the repository's clone URL. On Github, this can be easily obtained by visiting the project's Github page and copying the clone URL from the right side of the page. For third-party or private git repositories, you can obtain the clone URL by asking the project team directly. Make a note of the clone URL, as you'll need it in subsequent steps.

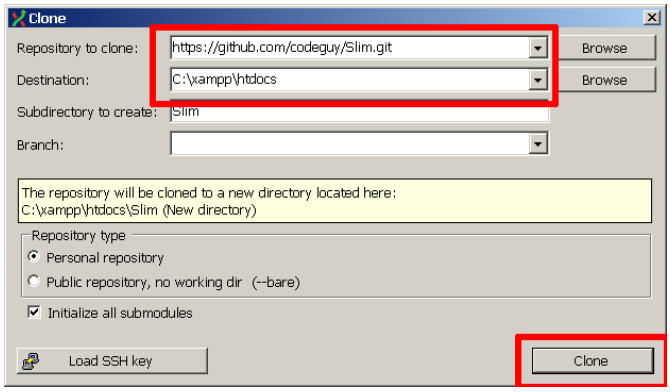


3. Launch Git Extensions by double-clicking the icon in the Windows Start menu or Start screen.
4. Click the "Clone repository" link or use the "Start _ Clone repository…_" menu item.

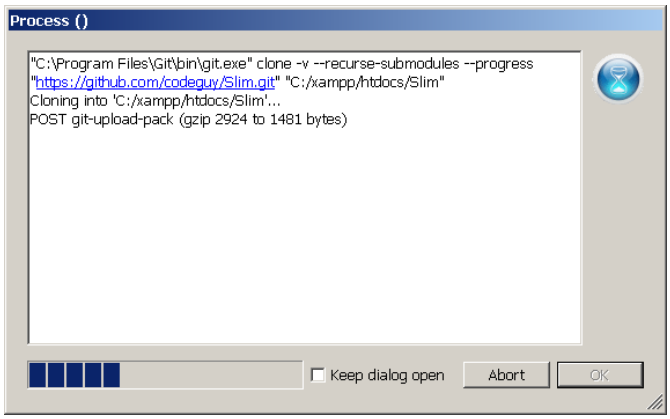


5. In the resulting "Clone" dialog box, enter the clone URL obtained in the first step. Set the destination directory to the *htdocs*\ subdirectory of your XAMPP installation directory (typically, *C:\xampp*). Optionally,

change the name of the subdirectory that will hold the downloaded code.

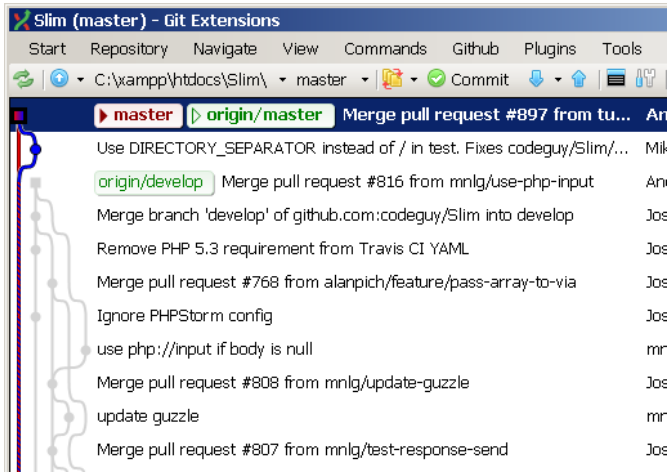


6. Click the "Clone" button to start cloning the repository. Git Extensions will display a progress dialog as it downloads the code from the source repository.

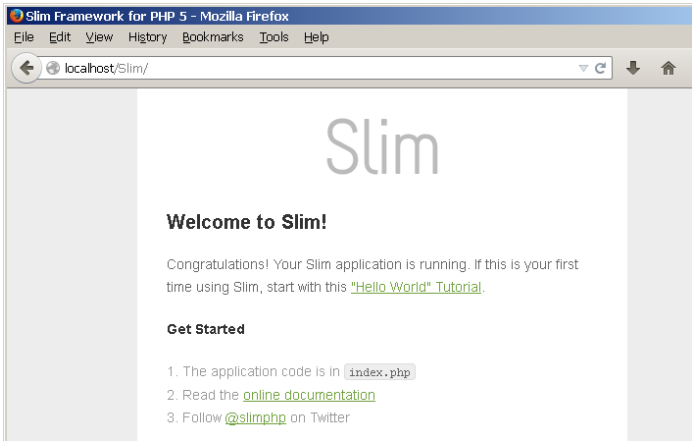


7. Once the repository has been cloned, Git Extensions will prompt you to open the new repository. Click "Yes".

   You should now see the entire commit history of the repository in a graphical tree structure, together with details on the changes made in each commit.



At this point, the code has been downloaded from the remote git repository and you can begin using it. In this particular case, since the Slim framework includes a starter application, you can simply browse to the URL http://localhost/Slim and you should see the example application's welcome page, as shown below.



NOTE

You can use Git Extensions to create a new branch of the codebase should you wish to make any changes to it. Any changes you make will be automatically reflected in the Git Extensions interface, and if you have appropriate privileges, you can also commit your changes back to the remote repository. Learn more about git, or read the Git Extensions documentation.

Increase the PHP File Upload Limit

To increase the allowed file size for uploads through PHP, follow these steps:

1. Edit the *php.ini* file in the *php/* subdirectory of your XAMPP installation directory (usually, *C:\xampp*). Within this file, find the *post_max_size* and *upload_max_filesize* variables and change them to reflect the new file upload limit. For example, to restrict uploads to not more than 16M, change these variables as shown below:

   ```
   ; Maximum size of POST data that PHP will accept.
   ; http://php.net/post-max-size
   post_max_size=16M
   ...
   ; Maximum allowed size for uploaded files.
   ; http://php.net/upload-max-filesize
   upload_max_filesize=16M
   ```

2. Save your changes to the file.

3. Restart the Apache server.

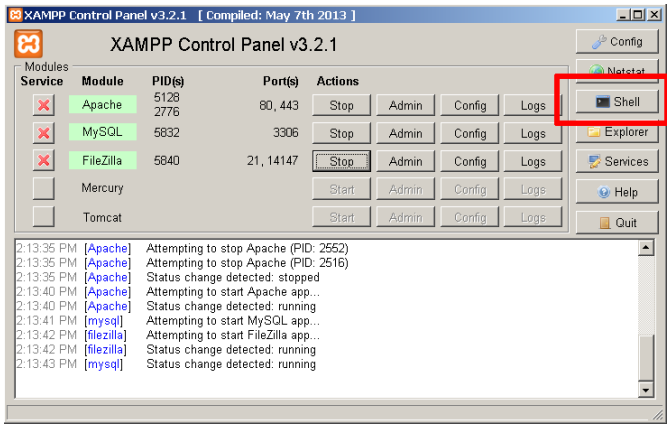
The new file upload limit will now come into effect.

Reset the MySQL/MariaDB Root Password

By default, the MySQL/MariaDB installation that ships with XAMPP has an empty root password. This is a serious security risk, especially if you plan to use XAMPP in production scenarios.

To change the MySQL/MariaDB root password, follow these steps:

1. Ensure that the MySQL/MariaDB server is running.
2. Open your Windows command prompt by clicking the "Shell" button in the XAMPP control panel.

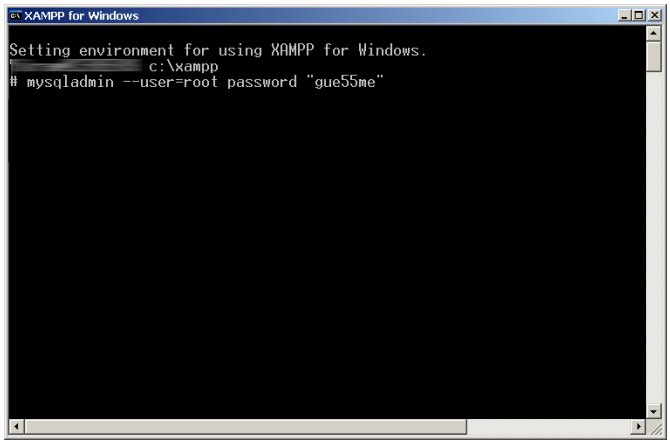


3. Use the *mysqladmin* command-line utility to alter the MySQL/MariaDB password, using the following syntax:

```
mysqladmin --user=root password "newpassword"
```

For example, to change the MySQL/MariaDB root password from its default empty value to the password *gue55me*, you would execute:

```
mysqladmin --user=root password "gue55me"
```



Or, if a password has already been previously set and you'd like to change it to a new one, you can use the following syntax:

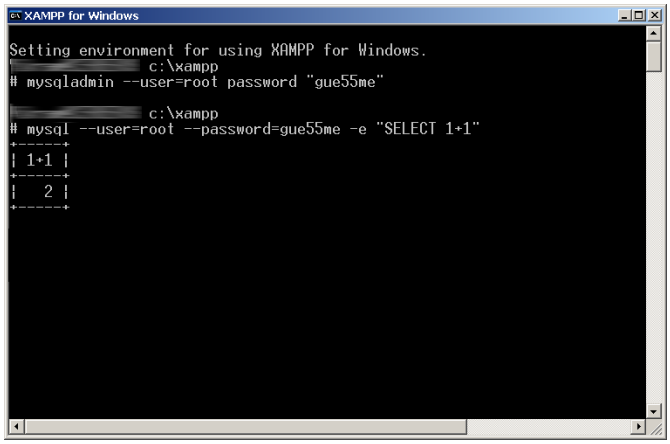
```
mysqladmin --user=root --password=oldpassword password "newpassword"
```

For example, to change the root password from *12345* to *gue55me*, you would execute:

```
mysqladmin --user=root --password=12345 password "gue55me"
```

4. Test that your password change has been accepted, by attempting to connect to the MySQL/MariaDB server using the *mysql* command-line client in the same directory. For example, you could use the command below to connect to the server and return the results of a calculation:

```
mysql --user=root --password=gue55me -e "SELECT 1+1"
```



Send Mail with PHP

If you're a PHP application developer, you might need to send email notifications through your custom PHP applications. With XAMPP, there are two ways to do this: you can use the included *sendmail.exe* binary, or you can use the third-party PHPMailer class. This guide walks you through both options.

NOTE

This guide assumes that you have an existing Gmail account, and that you will use Gmail's SMTP server to transmit email. You can also use your organization or ISP's mail server, although you'll need to replace key details, such as the SMTP server name and port, with appropriate values for your environment. Your network administrator or ISP support desk will typically be able to provide this information. The example code provided in this tutorial is illustrative only and should not be used in production environments.

To configure XAMPP to use *sendmail.exe* for email notifications, follow these steps:

1. Edit the *php.ini* file in the *php*\ subdirectory of your XAMPP installation directory (usually, *C:\xampp*). Within this file, find the [mail function] section and replace it with the following directives:

   ```
   sendmail_path = "\"C:\xampp\sendmail\sendmail.exe\" -t"
   ```

2. Edit the *sendmail.ini* file in the *sendmail*\ subdirectory of your XAMPP installation directory. Within this file, find the [sendmail] section and replace it with the following directives:

   ```
   smtp_server=smtp.gmail.com
   smtp_port=465
   smtp_ssl=auto
   error_logfile=error.log
   auth_username=your-gmail-username@gmail.com
   auth_password=your-gmail-password
   ```

   Remember to replace the dummy values shown with your actual Gmail address and account password.

3. Restart the Apache server using the XAMPP control panel.

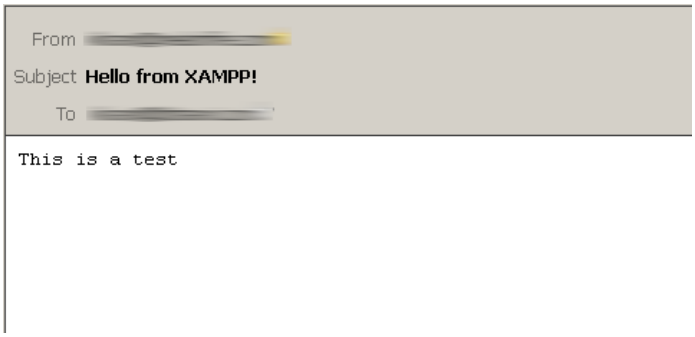
You can now use PHP's *mail()* function to send email from your application. To illustrate how it can be used with your Gmail account, use your text editor to create an example script named *sendmail.php* in the *htdocs*\ subdirectory and fill it with the following code. Once done, save your changes.

```
<?php
$to = 'recipients@email-address.com';
$subject = 'Hello from XAMPP!';
$message = 'This is a test';
$headers = "From: your@email-address.com\r\n";
if (mail($to, $subject, $message, $headers)) {
   echo "SUCCESS";
} else {
   echo "ERROR";
}
```

Remember to replace the dummy values shown with valid email addresses. For this simple test, use your own

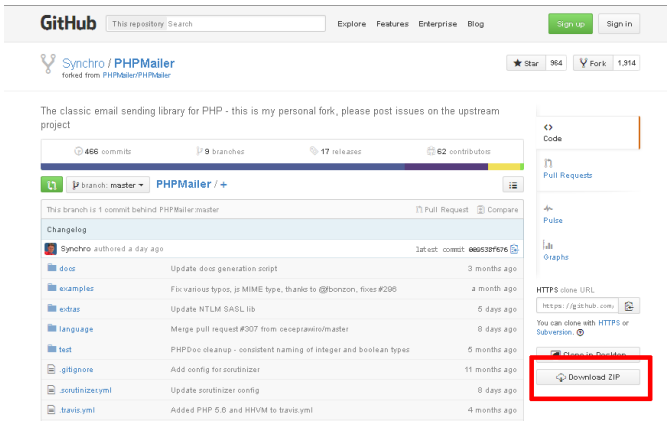
email address as the recipient address.

Now, browse to the URL http://localhost/sendmail.php to execute the script and send the email message. If all goes well, you should see a success notification in your browser. If you used your own email address for the recipient address, you should also receive the email message.



To configure XAMPP to use PHPMailer for email notifications, follow these steps:

1. Download PHPMailer from its Github repository using the "Download Zip" button.



2. Create a directory for your new application within the *htdocs*\ subdirectory of your XAMPP installation directory. In this tutorial, the application directory is named *example*\.

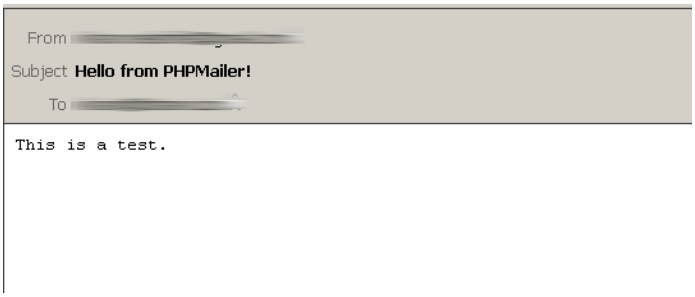
3. Extract the contents of the PHPMailer ZIP archive to the application directory.

You can now use PHPMailer to send email from your application. To illustrate how it can be used with your Gmail account, use your text editor to create an example script named *phpmailer.php* in the application directory, and fill it with the following code. Once done, save your changes.

```
<?php
require 'PHPMailer-master/PHPMailerAutoload.php';
$mail = new PHPMailer;
$mail->isSMTP();
$mail->SMTPSecure = 'ssl';
$mail->SMTPAuth = true;
$mail->Host = 'smtp.gmail.com';
$mail->Port = 465;
$mail->Username = 'your-gmail-username@gmail.com';
$mail->Password = 'your-gmail-password';
$mail->setFrom('your@email-address.com');
$mail->addAddress('recipients@email-address.com');
$mail->Subject = 'Hello from PHPMailer!';
$mail->Body = 'This is a test.';
//send the message, check for errors
if (!$mail->send()) {
    echo "ERROR: " . $mail->ErrorInfo;
} else {
    echo "SUCCESS";
}
```

Remember to replace the dummy values shown with your actual Gmail address and account password. You should also use a valid sender and recipient address. For this simple test, use your own email address as the recipient address.

Now, browse to the URL http://localhost/example/phpmailer.php. This should execute the script and send the email message. If all goes well, you should see a success notification in your browser. If you used your own email address for the recipient address, you should also receive the email message.



NOTE

As a security precaution, Gmail will automatically rewrite the From: and Reply-to: headers in your email message to reflect your Gmail address. If you want to avoid this, you must add and validate your custom email address in your Gmail account as a sender. Refer to Gmail's documentation for more information and important restrictions.

Configure FTP Access

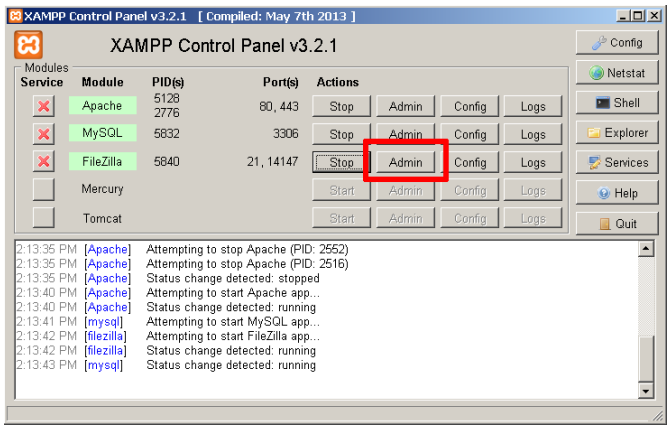
XAMPP includes the server version of FileZilla, an open-source FTP solution. This makes it easy to transfer files to and from a XAMPP environment using FTP.

To illustrate, assume that you have a simple PHP script named *example.php* on your Windows desktop containing the following code, that you wish to transfer to the XAMPP server.

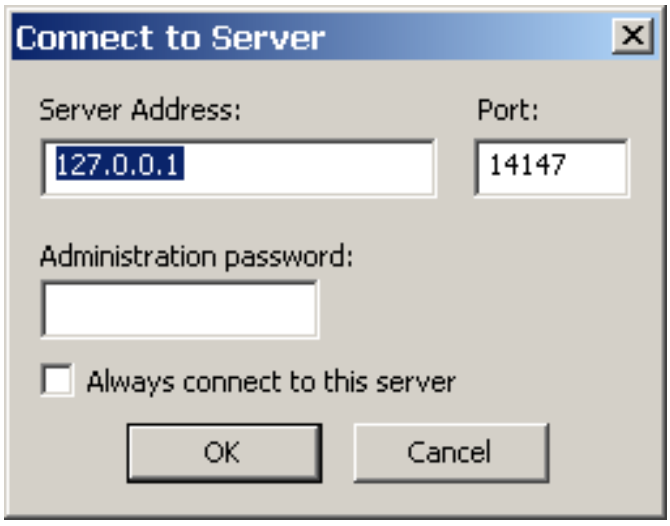
```
<!-- example.php -->
<html>
 <head></head>
 <body>
   <h2><?php echo "Hello. Today is " . date('l'); ?>.</h2>
 </body>
</html>
```

To transfer files using FTP, you must first configure the FileZilla FTP server. Follow the steps below.

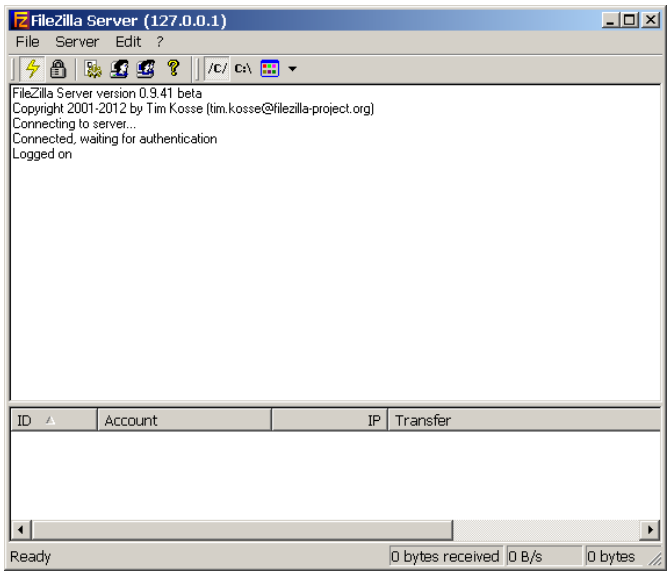
1. Ensure that FileZilla is running.
2. Start the FileZilla administration interface by clicking the "Admin" button in the XAMPP control panel.



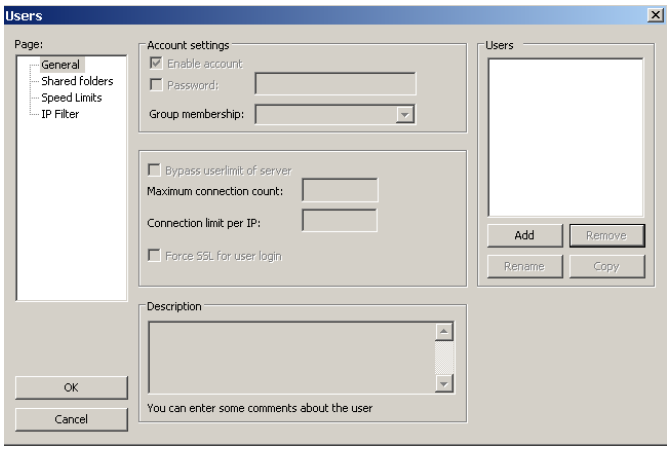
3. In the resulting "Connect to Server" dialog box, leave all values at their default values and click "OK".



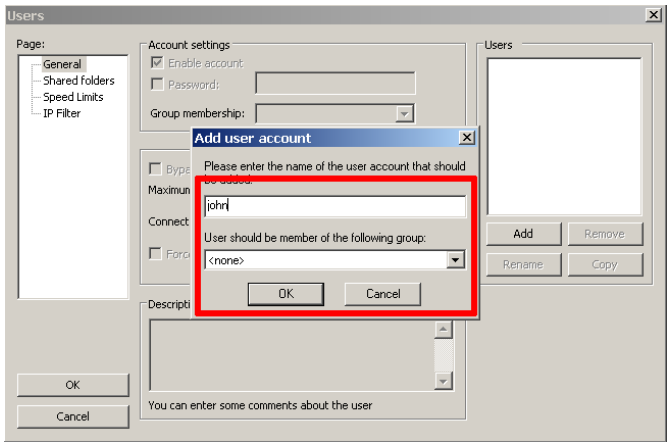
You should now arrive at the FileZilla server console.



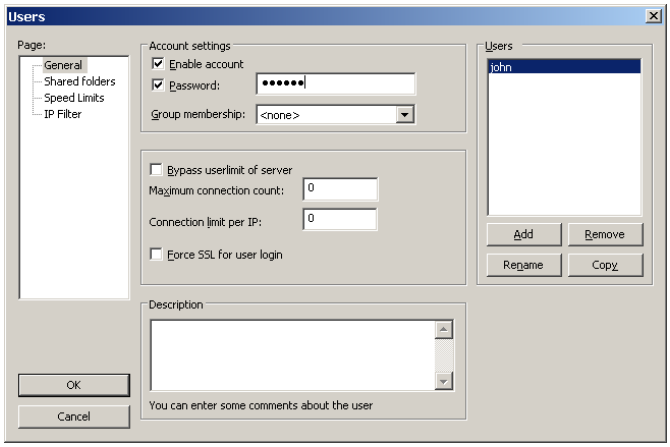
4. You now need to set up at least one FTP user. To do this, click the "Edit _ Users" menu item. This will bring you to the user management interface.



5. Click the "Add" button to add a new user account. Enter a name for the new user account.

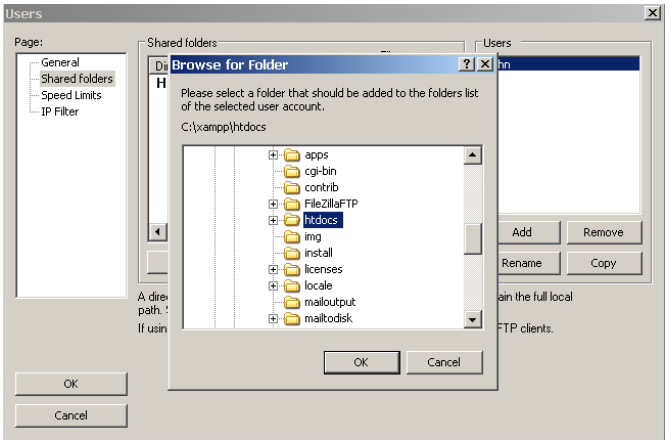


6. Select and check the "Password" field and enter a password for the new user.

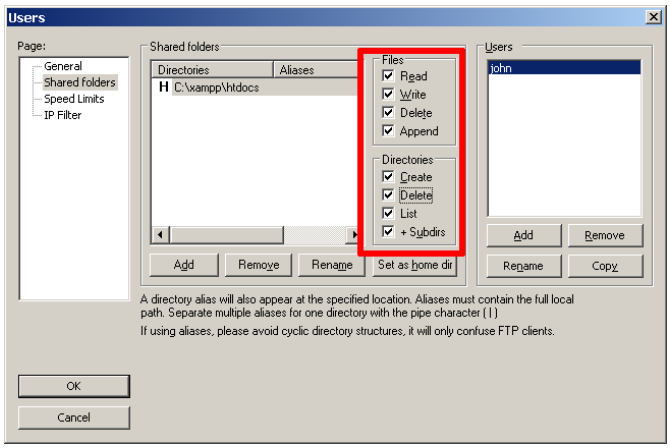


7. Next, you need to define which folders the user has access to. To do this, click the "Shared folder" sub-item in the left navigation menu.

8. Click the "Add" button and select the *htdocs*\ subdirectory of the XAMPP installation directory as the user's home directory.



9. Grant the user all available rights to this directory.



10. Click "OK" to save your changes.

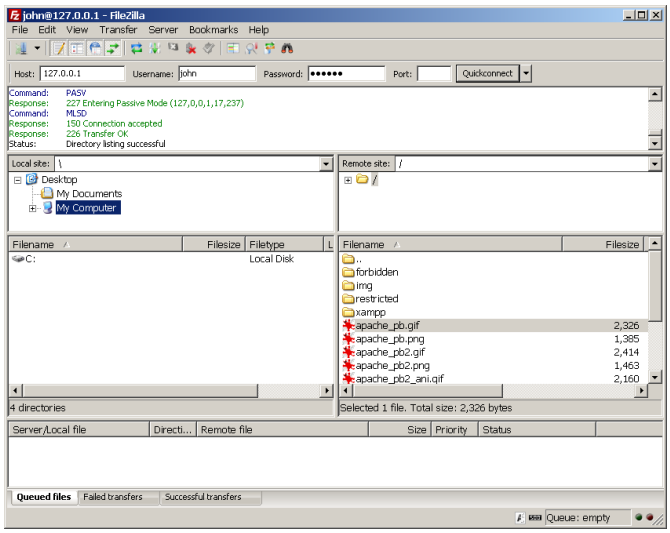
11. Use the "File _ Quit" menu item to disconnect from the FileZilla administration interface.

You can now transfer files to the XAMPP server by following the steps below:

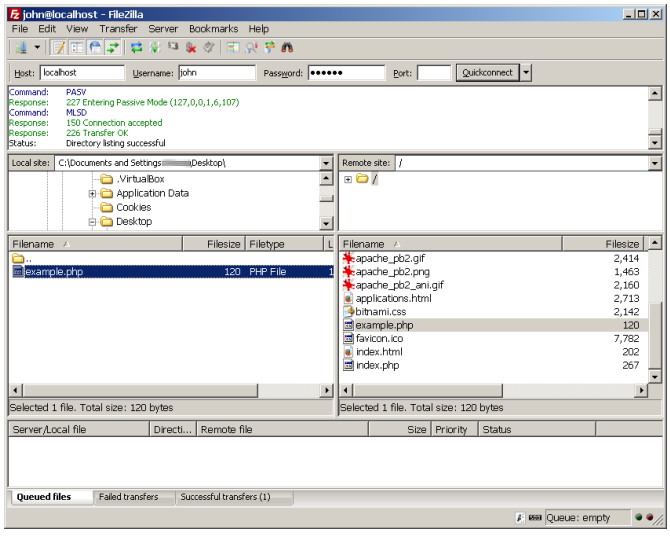
1. Launch your FTP client and enter connection details as below.

    - If you're connecting to the server from the same system, use "127.0.0.1" as the host address. If you're connecting from a different system, use the network hostname or IP address of the XAMPP server.
    - Use "21" as the port.
    - Specify the username and password configured previously as your FTP credentials.

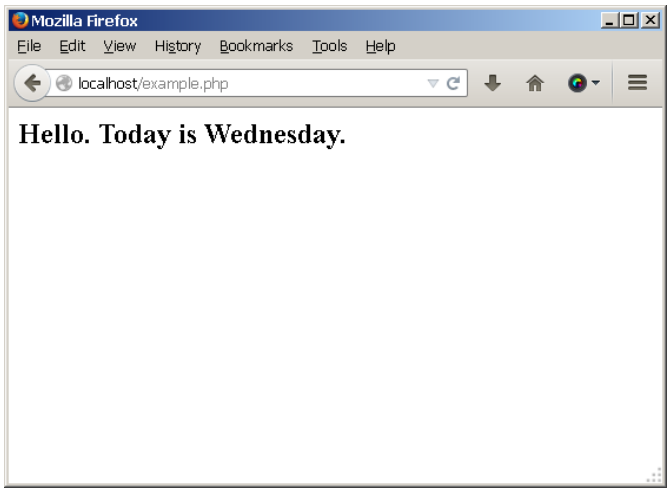
    Your FTP client should now connect to the server and enter the *C:\xampp\htdocs* directory, which is the default Web server document root.



2. Transfer the file from your home directory to the server using normal FTP transfer conventions. If you're using a graphical FTP client, you can usually drag and drop the file from one directory to the other. If you're using a command-line FTP client, you can use the FTP PUT command.



3. Once the file is successfully transferred, you should be able to see it in action by browsing to http://localhost/example.php, as shown below:


Mozilla Firefox
File Edit View History Bookmarks Tools Help
localhost/example.php
Hello. Today is Wednesday.

Troubleshoot Apache Startup Problems

Are you having difficulty getting your Apache server started? Here is a list of common problems and their solutions.

> **NOTE** The information in this document is based on this ApacheFriends community forum thread.

Port 80 in use by PID 4

Your XAMPP control panel displays this message:

```
XX:XX:XX [Apache] Port 80 in use by "Unable to open process" with PID 4!
```

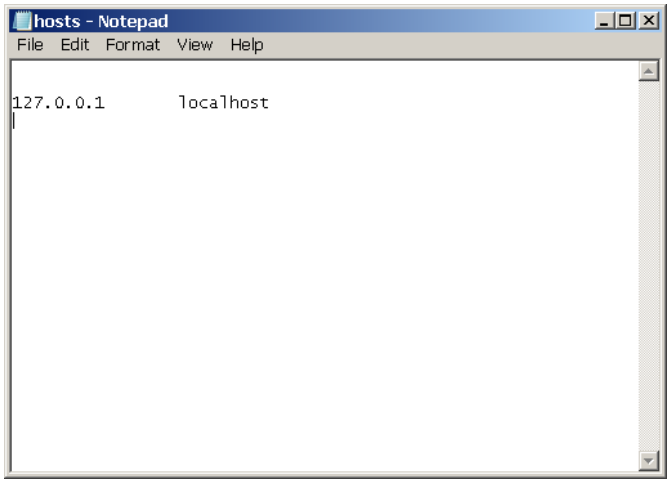
To solve this problem, follow these steps:

1. Right-click the XAMPP control panel icon and select "Run as administrator". Enter administrator credentials (if required) and click "OK" or "Yes" to proceed.
2. Open your Windows command prompt by clicking the "Shell" button in the XAMPP control panel.
3. At the command prompt, enter this command:

   ```
   netsh http add iplisten ipaddress=::
   ```

4. Open the file *C:\Windows\system32\drivers\etc\hosts* and add the following line to it. Then, save the file.

   ```
   127.0.0.1  localhost
   ```

   This takes care of mapping the *localhost* domain name to the local machine, which always has the IP address 127.0.0.1.

NOTE

On some versions of Windows, you will not be able to edit the *C:\Windows\system32\drivers\etc\hosts* file without administrator privileges. You can edit the file by right-clicking the Notepad icon and selected the "Run as administrator" menu option, then entering administrator credentials (if required) and clicking "OK" or "Yes" to proceed.

TIP

If the problem persists even after performing these steps, it could be due to a problem with Microsoft Internet Information Services (IIS), with Windows Work Folders or with third-party services. Refer to this page for alternative solutions.

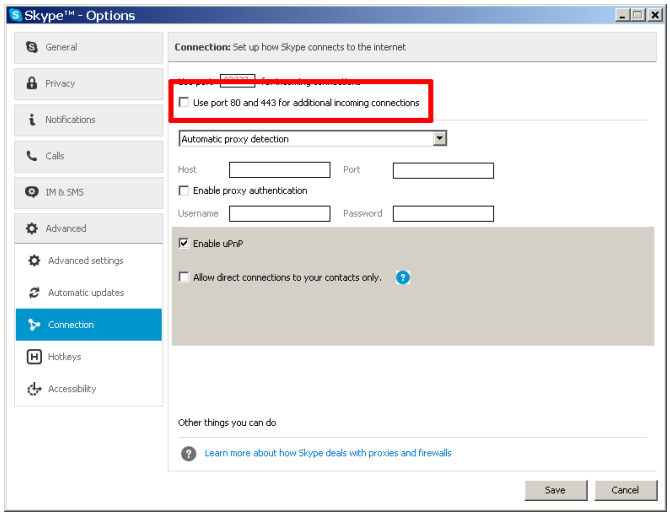
Ports 80 and 443 in use by Skype

Your XAMPP control panel displays these messages:

```
XX:XX:XX [Apache] Port 80 in use by "C:\Program Files (x86)\Skype\Phone\Skype.exe" with PID XXXX!
XX:XX:XX [Apache] Port 443 in use by "C:\Program Files (x86)\Skype\Phone\Skype.exe" with PID XXXX!
```

To solve this problem, follow these steps:

1. Launch Skype.
2. Select the "Tools _ Options _ Advanced _ Connection" menu item and in the resulting screen, disable the use of ports 80 and 443 for incoming Skype connections.



3. Click "Save". Restart Skype for your change to take effect.

Port 443 in use by VMWare

Your XAMPP control panel displays these messages:

```
XX:XX:XX [Apache] Port 443 in use by ""C:\Program Files (x86)\VMware\VMware Workstation\vmware-hostd.exe" -u "C:\ProgramData\VMware\hostd\config.xml"" with PID XXXX!
```

To solve this problem, follow these steps:

1. Launch VMWare Workstation.
2. Use the "Edit _ Preferences _ Shared VMs" menu and in the resulting dialog, enter a new port number for the VMWare Workstation Server.
3. Click "OK" to save your changes. Restart VMWare Workstation for your change to take effect.

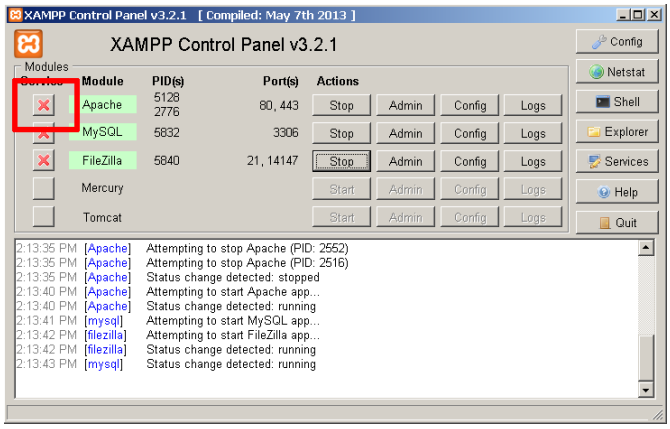
ServerRoot Must Be a Valid Directory

Apache startup includes an error message like the one below:

```
httpd.exe: Syntax error on line XX of X:/xampp/apache/conf/httpd.conf: ServerRoot must be a valid directory
```

This is a common issue for XAMPP installations on portable devices. It occurs if the Apache configuration file points to a directory that no longer exists (for example, due to a drive letter change when a portable device is removed and reinserted). To solve this problem, follow these steps:

1. If the Apache service is registered (indicated by a green check mark next to its name in the XAMPP control panel), unregister it by clicking the check mark and answering "Yes" in the resulting dialog box.



2. Edit the *setup_xampp.bat* file in your XAMPP installation directory (usually, *C:\xampp*). Within the file, find this line:

```
%PHP_BIN% -n -d output_buffering=0 %CONFIG_PHP%
```

3. Edit the line to include the special directive "usb" at the end, so that it looks like this.

```
%PHP_BIN% -n -d output_buffering=0 %CONFIG_PHP% usb
```

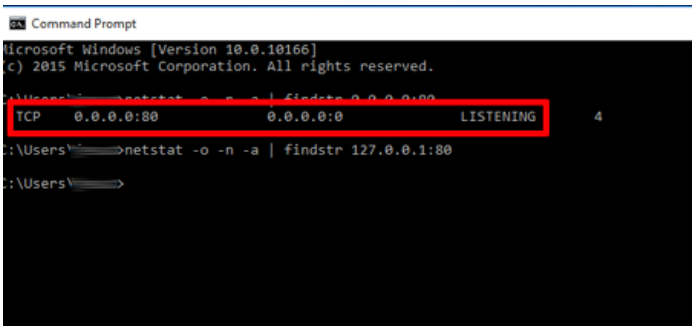
4. Save your changes. The, execute the file by double-clicking it in the Windows Explorer or executing it at the command prompt. This will fix any and all wrong base paths.

XAMPP cannot start on port 80 (Windows 10)

By default, Windows 10 starts Microsoft IIS on port 80, which is the same default port used by Apache in XAMPP. As a result, Apache cannot bind to port 80.

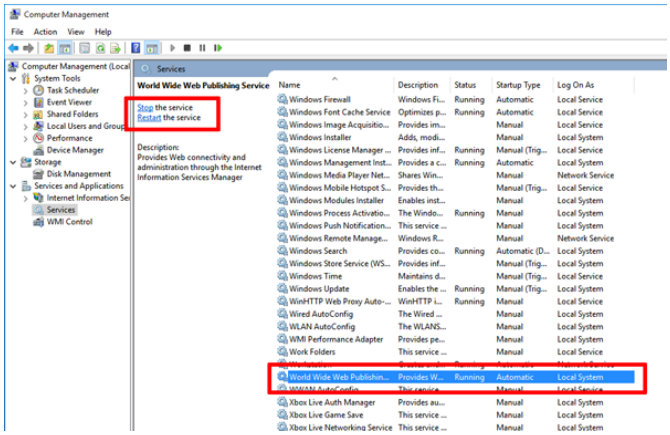
To check whether port 80 is in use, open a command prompt and run these commands:

```
netstat -o -n -a | findstr 0.0.0.0:80
netstat -o -n -a | findstr 127.0.0.1:80
```

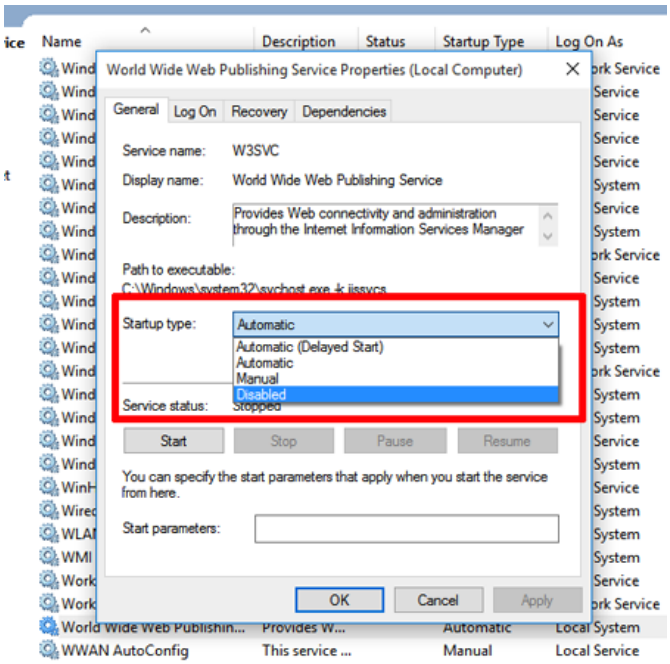


To deactivate IIS from running on port 80, follow these steps:

1. Open the Services panel in Computer Management.
2. Search for the "World Wide Web Publishing Service" and select it.
3. Click the link to "Stop the service".



4. Double-click the service name.
5. In the "Startup type" field, change the startup type to "Disabled".

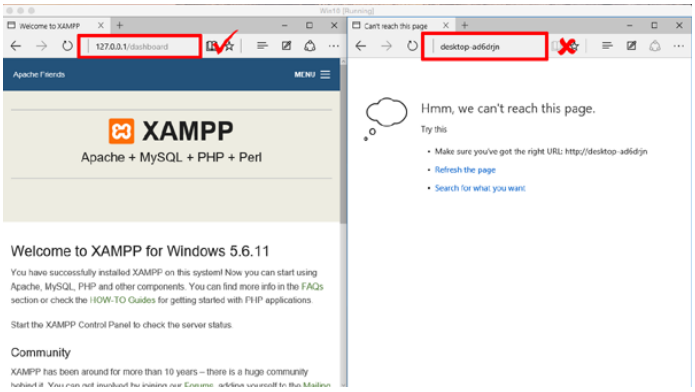


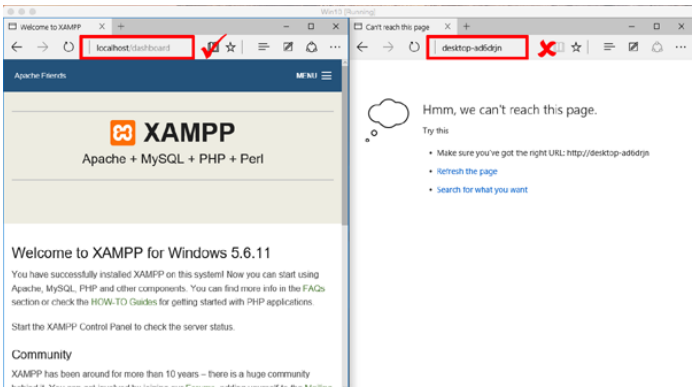
6. Click "OK" to save your changes.

You should now be able to start Apache in XAMPP on port 80.

The Microsoft Edge browser cannot be used to access local addresses using the machine host name.

If your local machine has the host name 'myhost', you will not be able to access URLs such as *http://myhost* in Microsoft Edge. To resolve this, you should instead use the addresses *http://127.0.0.1* or *http://localhost*.




Welcome to XAMPP
localhost/dashboard
Apache Friends
MENU
XAMPP
Apache + MySQL + PHP + Perl
Welcome to XAMPP for Windows 5.6.11
You have successfully installed XAMPP on this system! Now you can start using Apache, MySQL, PHP and other components. You can find more info in the FAQs section or check the HOW-TO Guides for getting started with PHP applications.
Start the XAMPP Control Panel to check the server status.
Community
XAMPP has been around for more than 10 years – there is a huge community
Can't reach this page
desktop-ad6drjn
Hmm, we can't reach this page.
Try this
Make sure you've got the right URL: http://desktop-ad6drjn
Refresh the page
Search for what you want

Use a Different PHP Version

In many cases, you might want to use XAMPP with a different PHP version than the one that comes pre-installed. You might do this to get the benefits of a newer version of PHP, or to reproduce bugs using an earlier version of PHP.

To use a different version of PHP with XAMPP, follow these steps:

1. Download a binary build of the PHP version that you wish to use from the PHP website, and extract the contents of the compressed archive file to your XAMPP installation directory (usually, *C:\xampp*). Ensure that you give it a different directory name to avoid overwriting the existing PHP version. For example, in this tutorial, we'll call the new directory *C:\xampp\php5-6-0*.

   NOTE Ensure that the PHP build you download matches the Apache build (VC9 or VC11) in your XAMPP platform.

2. Within the new directory, rename the *php.ini-development* file to *php.ini*. If you prefer to use production settings, you could instead rename the *php.ini-production* file to *php.ini*.

3. Edit the *httpd-xampp.conf* file in the *apache\conf\extra* subdirectory of your XAMPP installation directory. Within this file, search for all instances of the old PHP directory path and replace them with the path to the new PHP directory created in Step 1.

   In particular, be sure to change the lines

   ```
   LoadFile "/xampp/php/php5ts.dll"
   LoadFile "/xampp/php/libpq.dll"
   LoadModule php5_module "/xampp/php/php5apache2_4.dll"
   ```

   to

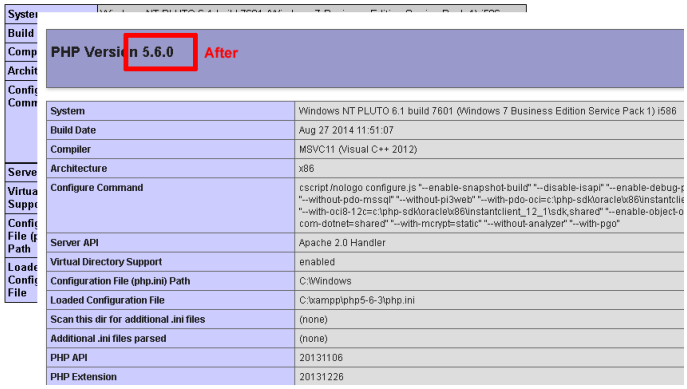
   ```
   LoadFile "/xampp/php5-6-0/php5ts.dll"
   LoadFile "/xampp/php5-6-0/libpq.dll"
   LoadModule php5_module "/xampp/php5-6-0/php5apache2_4.dll"
   ```

   NOTE Remember to adjust the file and directory paths above to reflect valid paths on your system.

4. Restart your Apache server through the XAMPP control panel for your changes to take effect.

The new version of PHP should now be active. To verify this, browse to the URL http://localhost/xampp/phpinfo.php, which displays the output of the *phpinfo()* command, and check the version number at the top of the page.





System	Windows NT PLUTO 6.1 build 7601 (Windows 7 Business Edition Service Pack 1) i586
Build Date	Aug 27 2014 11:51:07
Compiler	MSVC11 (Visual C++ 2012)
Architecture	x86
Configure Command	cscript /nologo configure.js "--enable-snapshot-build" "--disable-isapi" "--enable-debug-p "--without-pdo-mssql" "--without-pi3web" "--with-pdo-oci=c:\php-sdk\oracle\x86\instantclie "--with-oci8-12c=c:\php-sdk\oracle\x86\instantclient_12_1\sdk,shared" "--enable-object-o com-dotnet=shared" "--with-mcrypt=static" "--without-analyzer" "--with-pgo"
Server API	Apache 2.0 Handler
Virtual Directory Support	enabled
Configuration File (php.ini) Path	C:\Windows
Loaded Configuration File	C:\xampp\php5-6-3\php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20131106
PHP Extension	20131226

Run PHP under FastCGI

By default, XAMPP runs PHP as an Apache handler. An alternative approach is to run PHP under FastCGI, which can produce better performance in certain scenarios. To do this, follow these steps:

1. Download a binary version of the *mod_fcgid* Apache module from the ApacheLounge website. This module is included in the *modules-2.4.x-win32-VC11.zip* archive.

   NOTE XAMPP for Windows only uses 32-bit Apache builds.

2. Extract the *mod_fcgid.so* file from the downloaded archive file and copy it to the *apache\modules* subdirectory of your XAMPP installation directory (usually, *C:\xampp*).

3. Edit the *httpd-xampp.conf* file in the *apache\conf\extra* subdirectory of your XAMPP installation directory. Within this file, make the following changes:

   a. Search for the line

   ```
   LoadModule php5_module "/xampp/php/php5apache2_4.dll"
   ```

   and modify it to read

   ```
   LoadModule fcgid_module modules/mod_fcgid.so
   ```

   b. Find and delete the following lines:

   ```
   <FilesMatch "\.php$">
     SetHandler application/x-httpd-php
   </FilesMatch>
   <FilesMatch "\.phps$">
     SetHandler application/x-httpd-php-source
   </FilesMatch>
   ```

   c. Add the following lines:

   ```
   FcgidInitialEnv PHPRC "c:/xampp/php"
   AddHandler fcgid-script .php
   FcgidWrapper "c:/xampp/php/php-cgi.exe" .php
   ```

   NOTE Remember to adjust the file and directory paths above to reflect valid paths on your system.

4. Restart your Apache server through the XAMPP control panel for your changes to take effect.

PHP should now be active using FastCGI. To verify this, browse to the URL http://localhost/xampp/phpinfo.php, which displays the output of the *phpinfo()* command, and check the "Server API" value near the top of the page.

PHP Version 5.5.15



System	Windows NT PLUTO 6.1 build 7601 (Windows 7 Business Edition Service Pack 1) i586
Build Date	Jul 23 2014 14:58:09
Compiler	MSVC11 (Visual C++ 2012)
Architecture	x86
Configure Command	cscript /nologo configure.js "--enable-snapshot-build" "--disable-isapi" "--enable-debug-pack" "--without-mssql" "--without-pdo-mssql" "--without-pi3web" "--with-pdo-oci=C:\php-sdk\oracle\x86\instantclient10\sdk,shared" "--with-oci8=C:\php-sdk\oracle\x86\instantclient10\sdk,shared" "--with-oci8-11g=C:\php-sdk\oracle\x86\instantclient11\sdk,shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--with-mcrypt=static" "--disable-static-analyze" "--with-pgo"
Server API	CGI/FastCGI
Virtual Directory Support	enabled
Configuration File (php.ini)	C:\Windows

Use SQLite and PHP

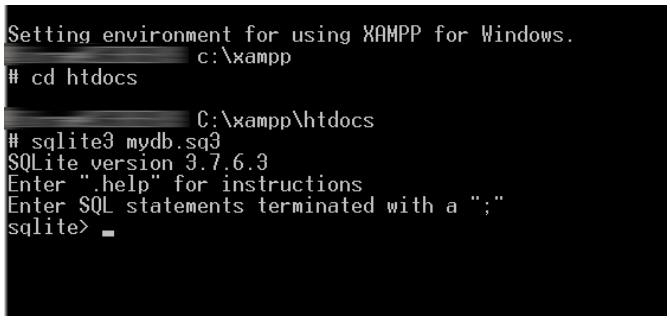
XAMPP comes with built-in support for SQLite, making it easy to get started building database-powered applications with PHP. This guide will walk you through the process of creating and populating a new SQLite database using XAMPP, then accessing the data within it using PHP.

Begin by creating and populating a new database using the SQLite command-line client, as below:

1. Open your Windows command prompt by clicking the “Shell” button in the XAMPP control panel.
2. Change to the *htdocs*\ subdirectory of your XAMPP installation directory (typically *C:**xampp*) and create a new SQLite database file named *mydb.sq3* with the SQLite command-line client:

   ```
   cd C:\xampp\htdocs
   sqlite3 mydb.sq3
   ```

   You should now see an SQLite prompt, as below:

3. At the *sqlite>* prompt, create a new table to hold your data. In this example, the table is named items and it contains 3 columns, for item ID, item name and item cost. You can use standard CREATE TABLE syntax to create the table, and you can learn more about SQLite’s built-in datatypes here.

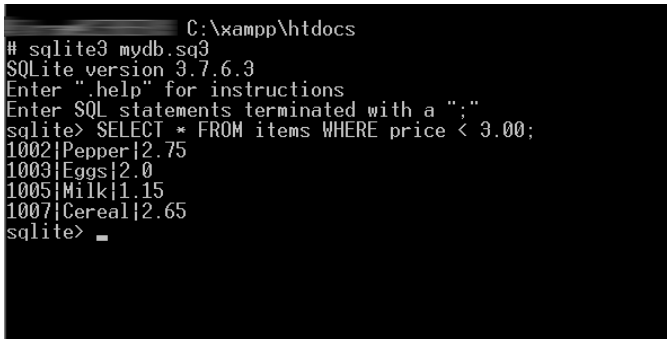
   ```
   CREATE TABLE items (id INTEGER PRIMARY KEY, name TEXT, price REAL);
   ```

4. Add some data to the new inventory table using INSERT commands, as shown below:

   ```
   INSERT INTO items VALUES ('1001', 'Salt', 3.15);
   INSERT INTO items VALUES ('1002', 'Pepper', 2.75);
   INSERT INTO items VALUES ('1003', 'Eggs', 2.00);
   INSERT INTO items VALUES ('1004', 'Bacon', 7.25);
   INSERT INTO items VALUES ('1005', 'Milk', 1.15);
   INSERT INTO items VALUES ('1006', 'Strawberries', 8.73);
   INSERT INTO items VALUES ('1007', 'Cereal', 2.65);
   ```

5. You can now also run a SELECT query on the data. For example, the query below returns all items that cost less than $3:

   ```
   SELECT * FROM items WHERE price < 3.00;
   ```



6. Once you're done using the database, exit it by typing *.quit* at the *sqlite>* prompt.

IMPORTANT	The database file (in this example, *mydb.sq3*) contains all your tables and data, so you should remember to back it up regularly.

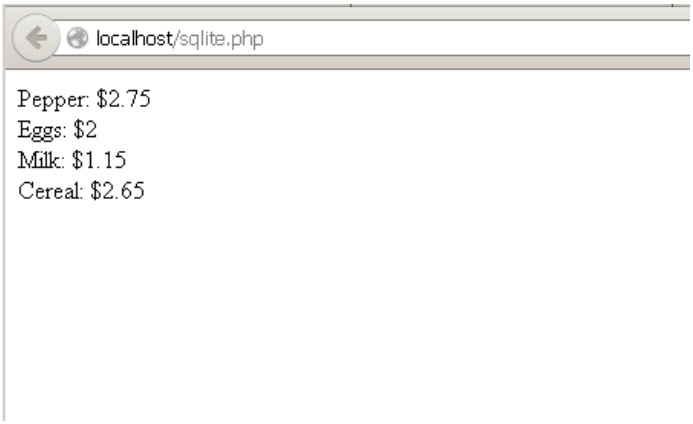
The previous steps discussed how to create and use an SQLite database using the command-line client. However, more often than not, you'll be using an SQLite database in combination with a PHP-powered Web application. XAMPP includes the PHP SQLite extension, so doing this is not very difficult at all.

To connect to your SQLite database and execute queries on it with PHP, use your text editor to create an example script named *sqlite.php* in the *htdocs* subdirectory of your XAMPP installation directory and fill it with the following code:

```
<?php
$db = new SQLite3('mydb.sq3');
$sql = "SELECT * FROM items WHERE price < 3.00";
$result = $db->query($sql);
while ($row = $result->fetchArray(SQLITE3_ASSOC)){
  echo $row['name'] . ': $' . $row['price'] . '<br/>';
}
unset($db);
```

The first line of code creates a new SQLite3 object, using the *mydb.sq3* database file you created earlier. Then, the object's *query()* method is used to execute a SELECT query on the database, and the result object's *fetchArray()* method is used to iterate over the result set. Adding the SQLITE3_ASSOC parameter to the *fetchArray()* method tells PHP to return the results as an associative array, making it easy to access individual fields of the result set and display them on a Web page.

Once done, save your changes and ensure that your Apache server is running. Then, browse to http://localhost/sqlite.php to execute the script. You should see something like this:



TIP To find out more about SQLite's powerful features, read the SQLite documentation.